

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет «Школа информатики, физики и технологий»

[Фамилия Имя Отчество автора]

**Адаптивные топологии мульти-агентных
систем больших языковых моделей с включением
человека в контур управления**

Выпускная квалификационная работа

бакалаврская работа

по направлению подготовки 01.03.02

«Прикладная математика и информатика»

образовательная программа

«Прикладной анализ данных и искусственный интеллект»

Рецензент

Руководитель

[уч. степень, звание]

[уч. степень, звание]

[И.О. Фамилия]

[И.О. Фамилия]

Санкт-Петербург, 2026

Оглавление

Аннотация	4
Abstract	4
Ключевые слова	5
Введение	6
1 Аналитический обзор литературы	10
2 Постановка задачи и методология исследования	16
3 Архитектура программной системы	26
4 Экспериментальное исследование	33
5 Анализ результатов и обсуждение	46
Авторский вклад	49
Заключение	52
Библиографический список	54
А Функциональная схема системы	57
Б Глоссарий	60
В Состав программного репозитория	62
Г Открытый исходный код	63

Аннотация

В работе проведена количественная оценка влияния выбора коммуникационной топологии мульти-агентной системы, механизма ее адаптации в процессе решения задачи и позиции человека в контуре управления на эффективность решения задач разных классов. Предмет исследования — коммуникационная топология мульти-агентных систем больших языковых моделей, механизм ее динамической адаптации и роль человека в графе обмена сообщениями; объект — сами мульти-агентные системы на основе больших языковых моделей. Разработан и опубликован под лицензией MIT (от англ. Massachusetts Institute of Technology) программный фреймворк, объединяющий пять статических топологий, адаптивную мета-топологию, пять ролей человека и три режима маршрутизации. Воронка из четырех экспериментов с двумя кросс-семейными подтверждениями охватывает 5175 запусков на четырех корпусах задач. Получены положительные ответы на два из четырех исследовательских вопросов; на два оставшихся — отрицательные. Помимо отрицательных ответов выявлены два сопутствующих наблюдения, которые при кросс-семейной валидации также не подтверждаются. Центральный вывод работы — кросс-семейная неустойчивость адаптивных политик маршрутизации, требующая обязательной валидации не менее чем на двух семействах моделей рабочего агента.

Abstract

The thesis provides a quantitative assessment of how the choice of communication topology, its runtime adaptation, and the position of the human in the control loop affect the effectiveness of multi-agent large-language-model systems on tasks of different classes. The subject of study is the communication topology of such systems, the mechanism of its dynamic adaptation, and the role of the human in the agent message graph; the object is multi-agent systems built on large

language models. A multi-agent framework was developed and published under the MIT license, integrating five static topologies, an adaptive meta-topology, five human roles, and three routing modes. A funnel of four experiments with two cross-family confirmation runs covers 5175 runs across four task corpora. Two of the four research questions receive positive answers; the remaining two are rejected. Alongside the negative answers, two related side findings are identified and likewise fail under cross-family validation. The central finding is the cross-family non-invariance of adaptive routing policies, which requires mandatory validation on at least two worker model families.

Ключевые слова

мульти-агентные системы, большие языковые модели, коммуникационная топология, адаптивная маршрутизация, человек в контуре управления, оракул адаптации, кросс-семейная валидация, Парето-оптимизация, бутстрап-оценивание, оркестрация LLM-агентов.

Keywords: multi-agent systems, large language models, communication topology, adaptive routing, human-in-the-loop, adaptation oracle, cross-family validation, Pareto optimization, bootstrap inference, LLM agent orchestration.

Введение

Актуальность работы. В 2023–2026 годах применение больших языковых моделей (от англ. Large Language Models, LLM) перешло от одиночных диалоговых сценариев к кооперативной работе нескольких автономных агентов, образующих мульти-агентную систему (от англ. Multi-Agent System, MAS). Открытые фреймворки (AutoGen, ChatDev, MetaGPT, Magentic-One) показывают, что разделение труда между специализированными агентами повышает качество решения сложных задач, но каждый из них жестко фиксирует структуру коммуникации на этапе проектирования; систематических данных об оптимальной топологии для каждого класса задач в открытой литературе нет. Параллельный интерес к интеграции человека в контур управления ограничивается единичными паттернами без сравнения эффективности. Сочетание этих обстоятельств определяет научную и прикладную актуальность работы.

Научная новизна. В настоящей работе впервые проведено прямое сравнение пяти базовых коммуникационных топологий (звезда, цепочка, полносвязная, дебатная, иерархическая) на едином программном стенде, на одной языковой модели и на одной выборке задач из четырех различных классов. Предложена и реализована мета-топология, переключающая активную базовую конфигурацию в зависимости от текущей фазы решения и наблюдаемых сигналов агентов. Введена таксономия из пяти специфичных для адаптивных мульти-агентных систем метрик — N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{oracle} , — позволяющая количественно характеризовать поведение мета-топологии. Впервые сформулирована и экспериментально проверена матрица совместимости пяти ролей человека в контуре управления с пятью базовыми топологиями.

Предмет исследования. Коммуникационная топология мульти-агентной системы больших языковых моделей, механизм ее динамической адаптации в процессе решения задачи, а также позиция человека в графе обмена сообщениями между агентами.

Объект исследования. Мульти-агентные системы на основе больших

языковых моделей, решающие сложные задачи различных классов в кооперации со специализированными агентами и участником-человеком.

Цель работы. Количественная оценка влияния выбора коммуникационной топологии мульти-агентной системы, механизма ее адаптации и позиции человека в контуре управления на эффективность решения задач разных классов с одновременным учетом качества решения, его стоимости и времени выполнения.

Аналитический обзор предлагаемых решений. В 2022–2026 годах выделяются три направления развития MAS LLM. Архитектурные фреймворки (AutoGen, ChatDev, MetaGPT) фиксируют топологию и не сравнивают альтернативы. Автоматическое проектирование графов (G-Designer, GPTSwarm) подбирает структуру однократно и не пересматривает ее. Динамическая адаптация (DyLAN, AgentVerse) меняет состав команды, но не структуру связей. Вопрос роли человека освещен фрагментарно, без количественной оценки ролей. Подробный анализ — в главе 1, сравнительная таблица 1.1 восемнадцати методов по шести критериям.

Практическая значимость. Работа дает разработчикам MAS количественные данные о том, какая топология эффективнее для каждого из четырех рассмотренных классов задач и когда имеет смысл адаптивное переключение. Программная инфраструктура реализована как открытый фреймворк адаптивных топологий (от англ. Adaptive Topologies for Multi-agent systems, ATM), исходный код — в репозитории GitHub (приложение Г), допускает повторное использование и расширение. Сформулированные правила выбора топологии и роли человека под класс задачи применимы при проектировании интеллектуальных ассистентов в программировании, аналитике и принятии решений.

Теоретическая значимость. Введенная таксономия адаптивно-специфичных метрик (N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{oracle}) позволяет количественно характеризовать поведение динамически перестраиваемых MAS по единой шкале. Формализация MAS как кортежа из шести компонентов с фазово-зависимой коммуникационной структурой расширяет теоретическую базу; эмпирическая

проверка гипотезы о фазовой специфике топологий дает обоснование для дальнейших теоретических работ.

Методы исследования. Комплекс взаимодополняющих методов: аналитический обзор литературы по ключевым словам в открытых базах; формализация MAS средствами теории графов и конечных автоматов; программная реализация на Python с LangGraph; контролируемый эксперимент с предварительной регистрацией гипотез и порогов улучшения; статистическая обработка (дисперсионный анализ от англ. Analysis of Variance, ANOVA; парные сравнения с поправкой Бенджамини–Хохберга на уровне ложных открытий от англ. False Discovery Rate, FDR; коэффициент Коэна; непараметрический бутстрап с доверительными интервалами от англ. Confidence Interval, CI); оракульный референс per-task best-of для верхней границы качества адаптации; имитационное моделирование участника-человека языковой моделью с подсказкой по роли.

Задачи работы. Для достижения сформулированной выше цели в работе решаются следующие задачи.

1. Провести аналитический обзор работ 2022–2026 годов по мульти-агентным системам больших языковых моделей, коммуникационным топологиям и интеграции человека в контур управления, выделить исследовательскую лакуну.
2. Сформулировать формальную модель мульти-агентной системы как кортежа из множества агентов, множества ребер коммуникации, множества ролей агентов и человека, задачи и множества фаз решения.
3. Разработать и реализовать программный фреймворк, в котором пять базовых топологий и адаптивная мета-топология реализованы в единой архитектуре на основе библиотеки LangGraph.
4. Спроектировать систему оценочных метрик, включающую базовые показатели качества, стоимости и времени, а также специфические для адаптивной мета-топологии характеристики переключений, заблокированных решений маршрутизатора, стоимостного вклада маршрутизатора, доли регрессий качества и разрыва до верхнего потолка

адаптации.

5. Провести воронку из четырех экспериментов и подтверждающего запуска на четырех классах задач — программирование, математические рассуждения, творческая генерация, анализ данных — с предварительно зафиксированными гипотезами и порогами улучшения.
6. Выполнить статистический анализ результатов и сформулировать ответы на четыре исследовательских вопроса (от англ. Research Question, RQ), описанных в разделе 2.7.
7. Выработать практические рекомендации по выбору топологии и роли человека в зависимости от класса задачи.

Структура работы. Работа состоит из введения, пяти глав основного содержания, раздела авторского вклада, заключения, библиографического списка и четырех приложений (А, Б, В, Г). Глава 1 — аналитический обзор литературы и исследовательская лакуна. Глава 2 — формальная модель, шесть топологий, пять ролей человека, метрики, корпус, дизайн воронки. Глава 3 — архитектура программной системы. Глава 4 — результаты четырех основных экспериментов и подтверждающих запусков на дополнительном семействе моделей. Глава 5 — анализ, ответы на четыре RQ и угрозы валидности. Раздел авторского вклада фиксирует оригинальные элементы; заключение подводит итоги и формулирует направления дальнейших исследований. Приложения — функциональная схема, глоссарий, состав репозитория, ссылка на исходный код.

В работе пять глав основного содержания, четыре приложения, 19 таблиц, 8 рисунков и 5 формул; общий объем работы с введением, заключением, библиографическим списком и приложениями составляет 62 страницы; библиографический список — 44 источника.

1 Аналитический обзор литературы

Глава охватывает пять направлений — архитектуру мульти-агентных систем больших языковых моделей, коммуникационные топологии, механизмы их адаптации, безопасность и интеграцию человека в контур управления — и завершается сравнительной таблицей методов и определением исследовательской лакуны.

В выборку включены работы 2022–2026 годов, отобранные по ключевым словам *multi-agent LLM*, *agent topology*, *LLM debate*, *human-in-the-loop multi-agent* в базах arXiv, ACL Anthology, ICLR, NeurIPS, ICML. Из более чем шестидесяти работ оставлены те, которые либо предлагают новую архитектуру взаимодействия агентов, либо систематически анализируют свойства существующих.

1.1 Мульти-агентные системы больших языковых моделей

С появлением больших языковых моделей (от англ. Large Language Models, LLM) концепция мульти-агентных систем (от англ. Multi-Agent System, MAS) переживает второе рождение — роль агента теперь играет языковая модель с системной подсказкой и набором инструментов. Обзор [1] систематизирует быстрорастущую совокупность исследований и выделяет несколько устойчивых архитектурных образцов.

Фреймворк AutoGen [4] реализует многораундовый диалог агентов с вызовом инструментов и подключением человека, но структура взаимодействия определяется императивным кодом сценария и не сравнивается между конфигурациями. ChatDev [8] моделирует разработку ПО как последовательность ролевых агентов (директор, программист, тестировщик) по водопадной модели; MetaGPT [31] развивает идею через стандартные операционные процедуры. В обеих системах топология жестко зашита в архитектуру. Одно-агентные методы Self-Refine [41] и Reflexion [38] реализуют цикл «генерация — критика — переработка» с вербальным подкреплением, но не используют разнообразия точек зрения мульти-агентных конфигураций.

1.2 Коммуникационные топологии агентов

Под коммуникационной топологией понимается граф, описывающий, какие агенты могут обмениваться сообщениями и в каком порядке. Таксономия настоящей работы включает пять базовых типов — звезда, цепочка, полностью связная, дебатная, иерархическая — изображенных на рисунке 1.1.

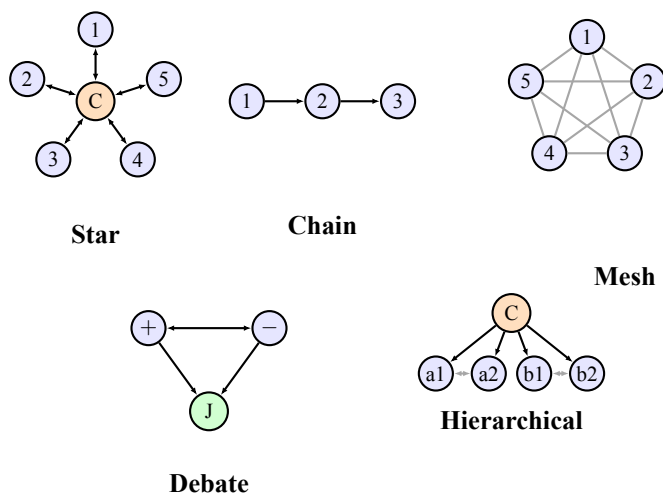


Рисунок 1.1. Схематическое изображение пяти базовых коммуникационных топологий, рассматриваемых в работе. Узлы — агенты; С — координатор, J — арбитр, + и — — оппоненты в дебатах, a1, a2, b1, b2 — участники подкоманд. Сплошные направленные ребра обозначают передачу управления, серые двунаправленные — обмен сообщениями в общей шине.

GPTSwarm [17] рассматривает мульти-агентную систему как обучаемый граф вычислений и оптимизирует его параметры на корпусе задач, но пространство поиска ограничено статичными структурами. G-Designer [16] применяет графовые нейронные сети (от англ. Graph Neural Network, GNN) для подбора топологии под задачу, но выбор производится однократно. В [40] показано, что топологии типа малого мира (small-world) повышают робастность, но сравнение ведется только между статичными конфигурациями. DyLAN [28] варьирует состав команды, оставляя структуру связей фиксированной; Mixture-of-Agents [32] реализует многослойную ансамблевую конфигурацию без изменения во время решения.

Tree-of-Thoughts [44] разворачивает дерево вариантов рассуждения, CAMEL [6] моделирует ролевой диалог двух агентов; дебаты языковых моделей [21] по-

вышают качество вывода через обмен аргументами. Во всех этих методах структура взаимодействия фиксирована на протяжении задачи. Возможность переключения топологии в процессе решения в этих работах не рассматривается.

1.3 Механизмы адаптации топологий

Адаптация мульти-агентных систем во время выполнения исследуется реже. Преобладают два направления, ни одно из которых не покрывает динамическое переключение топологии между фазами.

Первое направление — адаптация состава команды. DyLAN [28] выбирает на каждом шаге подмножество агентов; AgentVerse [3] использует механизм найма из заранее заданного пула. Изменяется множество участников, а не схема обмена. Второе направление — однократная офлайн-оптимизация структуры графа перед запуском задачи (GPTSwarm [17], G-Designer [16]).

Сложные задачи неоднородны по структуре — фаза планирования требует генерации альтернатив, исполнения — иерархического разделения труда, проверки — независимой рецензии. Отсюда гипотеза: статическая топология, оптимальная для усредненной картины, может быть субоптимальной для каждой отдельной фазы. Эта гипотеза мотивирует разработку механизма переключения топологии в процессе выполнения, что и составляет один из центральных предметов исследования.

1.4 Безопасность и устойчивость топологий

NetSafe [35] показывает, что плотные топологии (полносвязная, звезда) более подвержены каскадному распространению ошибок при компрометации агента, чем разреженные (цепочка). ТОМА [42] развивает анализ на многошаговые атаки с учетом связности. Эти результаты мотивируют включение в систему защитных правил переключения топологий, предотвращающих патологические режимы (в частности, осцилляционное переключение без прогресса); их реализация описана в главе 3.

1.5 Человек в контуре управления

Подключение человека (от англ. Human-in-the-Loop, HITL) систематизировано в [34] в части разметки, исправления и активного отбора; классическая работа [20] ввела принципы смешанной инициативы. Методы выравнивания моделей с обратной связью от человека ([10], InstructGPT [36]) ставят человека в роль поставщика сигнала вознаграждения — в MAS вопрос принципиально иной — роль человека в каждом вызове требует формализации в графе сообщений.

Обзор [1] перечисляет паттерны (координатор, рецензент, судья), но без сравнения эффективности. AutoGen [4] допускает человека как агента, но роль задается архитектурно. Magentic-One [29] разделяет координатора и работников с опциональным человеком-постановщиком задачи, также без сравнения ролей. Шкала NASA-TLX [18] применяется в оценке когнитивной нагрузки, но не пересекается с проектированием графа агентов.

В выборке 2022–2026 годов систематического сравнения ролей человека и фаз их активации не обнаружено — эта лакуна составляет второй центральный предмет исследования.

1.6 Сравнительная аналитическая таблица методов

Сопоставление методов по ключевым критериям приведено в таблице 1.1. Знаки «+» / «−» / «±» означают наличие, отсутствие и частичную реализацию свойства.

1.7 Что отсутствует в существующих работах

Три столбца таблицы 1.1 — систематическое сравнение топологий, адаптация во время решения и формализованная роль человека — в выборке не встречаются одновременно. G-Designer [16] сравнивает топологии, но не адаптирует их в процессе; AutoGen [4] подключает человека без формализации и сравнения ролей; DyLAN [28] адаптирует состав, а не структуру связей.

Заполнение этой лакуны составляет цель работы. Предлагаются — во-

Таблица 1.1. Сравнение существующих методов по ключевым критериям

Метод	Год	MAS	Сравн. топ.	Адапт. состава	Адапт. топ. runtime	HITL	Безопасн.
ReAct [37]	2022	—	—	—	—	—	—
AgentVerse [3]	2023	+	—	+	—	—	—
LLM debate [21]	2023	+	—	—	—	—	—
CAMEL [6]	2023	+	—	—	—	—	—
DyLAN [28]	2023	+	—	+	—	—	—
Reflexion [38]	2023	—	—	—	—	—	—
AutoGen [4]	2023	+	—	—	—	+	—
Tree-of-Thoughts [44]	2023	—	—	—	—	—	—
Magentic-One [29]	2024	+	—	—	—	±	—
MetaGPT [31]	2024	+	—	—	—	—	—
TOMA [42]	2024	+	±	—	—	—	+
Self-Refine [41]	2024	—	—	—	—	—	—
ChatDev [8]	2024	+	—	—	—	—	—
Scaling MAS [40]	2024	+	±	—	—	—	±
MoA [32]	2024	+	—	—	—	—	—
NetSafe [35]	2024	+	+	—	—	—	+
G-Designer [16]	2024	+	+	—	—	—	—
GPTSwarm [17]	2024	+	±	—	—	—	—
Настоящая работа	2026	+	+	—	+	+	±

Обозначения колонок — **MAS** мульти-агентная архитектура, **Сравн. топ.** систематическое сравнение нескольких топологий на едином стенде, **Адапт. состава** динамическое изменение участников, **Адапт. топ. runtime** переключение структуры графа в процессе решения задачи, **HITL** интеграция человека в контур управления как полноценного участника графа, **Безопасн.** учет устойчивости к скомпрометированному агенту и каскадным сбоям. Знак «+» означает наличие свойства, «—» — его отсутствие, «±» — частичную реализацию.

первых, единый стенд для сравнения пяти топологий на четырех классах задач; во-вторых, мета-топология с переключением активной базовой по фазе решения; в-третьих, формализация пяти ролей человека с экспериментальной оценкой по факторному плану.

1.8 Выводы по главе

Коммуникационная топология в существующих MAS фиксируется на этапе проектирования; механизмы адаптации ограничены составом команды или однократным офлайн-выбором структуры. Безопасность существенно зависит от топологии, что мотивирует защитные правила при ее динамическом изменении. Систематического исследования роли человека в графе агентов не обнаружено; смежные работы по бенчмаркингу (AgentBench [2]) и маршрутизации между моделями (RouteLLM [39], FrugalGPT [9]) формализованной роли человека также не вводят. На пересечении трех направлений

— сравнения, адаптации, роли человека — лежит лакуна, формализуемая четырьмя исследовательскими вопросами в разделе 2.

2 Постановка задачи и методология исследования

Глава формализует объект исследования, исследовательские вопросы и методологию их эмпирической проверки — математическая модель, шесть топологий, пять ролей человека, система метрик, обоснование корпуса и моделей, воронка из четырех экспериментов.

2.1 Формальная модель мульти-агентной системы

Мульти-агентная система рассматривается как кортеж

$$\mathcal{M} = \langle V, E_t, R, \mathcal{T}, F, h \rangle, \quad (2.1)$$

где $V = \{v_1, \dots, v_n\}$ — конечное множество агентов, каждый из которых реализован как языковая модель с фиксированной системной подсказкой и набором инструментов; $E_t \in 2^{V \times V}$ — множество допустимых направленных ребер коммуникации на итерации t , образующее коммуникационный граф; $R: V \rightarrow \mathcal{R}_A$ — отображение агентов в множество ролей агентов $\mathcal{R}_A$ из шести значений (планировщик, исследователь, исполнитель, критик, дебатер, координатор); $\mathcal{T}$ — текущая задача с целевой функцией оценки качества решения; F — конечное множество из четырех фаз решения задачи (планирование, исполнение, проверка, завершение); h — (опционально) участник-человек с ролью из отдельного множества $\mathcal{R}_H$ (см. 2.3).

В классических работах граф E_t определяется архитектурно и не меняется в ходе выполнения задачи. В настоящем исследовании вводится расширение модели — структура E_t становится результатом оператора обновления, зависящего от текущей фазы $f_t \in F$ и наблюдаемых сигналов $\sigma_t \in \{0, 1\}^m$ агентов:

$$E_t = \Phi(f_t, \sigma_t, t), \quad (2.2)$$

где Φ — оператор обновления структуры коммуникационного графа, m —

фиксированная размерность вектора сигналов (флаги завершения этапа, признаки тупикового состояния, счетчики отказов критика и другие наблюдаемые индикаторы прогресса). Возможность такого переключения и составляет суть исследуемой адаптивной мета-топологии.

Множество ролей агентов $\mathcal{R}_A$ выше уже зафиксировано шестью значениями. Множество ролей человека $\mathcal{R}_H$ фиксировано и содержит пять значений, формализованных в разделе 2.3. Фазы F упорядочены отношением $planning \prec execution \prec verification \prec done$ с монотонной семантикой переходов.

Задача оптимизации мульти-агентной системы формулируется как многокритериальная по двум первичным осям — качеству решения $q(\mathcal{M})$ и суммарной стоимости вызовов $c(\mathcal{M})$ — при ограничении на время выполнения $\tau(\mathcal{M}) \leq \tau_{\max}$, где $\tau_{\max}$ задается конфигурацией эксперимента. Поскольку чистый Парето-оптимум обычно не единственен, в работе используется скаляризация с фиксированными порогами улучшения по двум первичным осям. Конфигурация $\mathcal{M}_1$ считается выигрышной относительно $\mathcal{M}_0$, если соблюдено ограничение $\tau(\mathcal{M}_1) \leq \tau_{\max}$ и выполняется одно из условий

$$\frac{q(\mathcal{M}_1) - q(\mathcal{M}_0)}{q(\mathcal{M}_0)} \geq 0,05 \quad \text{или} \quad \frac{c(\mathcal{M}_0) - c(\mathcal{M}_1)}{c(\mathcal{M}_0)} \geq 0,15 \quad \text{при} \quad q(\mathcal{M}_1) \geq q(\mathcal{M}_0). \quad (2.3)$$

Пороги 5% для качества и 15% для стоимости зафиксированы до проведения основных запусков и обоснованы в разделе 2.4.

2.2 Шесть рассматриваемых топологий

Пространство сравнения составляют шесть топологий. Пять из них являются статичными и реализуют пять базовых классов коммуникационных структур, перечисленных в обзоре литературы. Шестая является мета-топологией, переключающей активную статическую топологию в зависимости от фазы.

Star — центральный координатор v_c последовательно обращается к специализированным работникам $v_1, \dots, v_k$ и агрегирует их ответы. Граф ребер $E_{\text{star}} = \{(v_c, v_i), (v_i, v_c) \mid i = 1, \dots, k\}$.

Chain — линейный конвейер из трех агентов $v_p \rightarrow v_e \rightarrow v_c$ (планировщик, исполнитель, критик) с условным циклом доработки. Граф направленный ациклический с обратной связью от критика к исполнителю.

Mesh — полносвязная топология с общей шиной сообщений. Граф $E_{\text{mesh}} = (V \times V) \setminus \{(v, v)\}$, обмен организован по принципу циклической очереди, решение принимается голосованием.

Debate — пара оппонентов v_+ и v_- генерирует параллельные альтернативные решения, после чего судья v_j выбирает победителя или синтезирует ответ. Граф двудольный с агрегатором.

Hierarchical — двухуровневая иерархия из координатора верхнего уровня v_c^{top} и двух подкоманд $T_a, T_b \subset V$, каждая из которых сама по себе является графом. Подкоманды работают параллельно, координатор синтезирует итог.

Adaptive — мета-топология, активирующая на каждой итерации одну из пяти базовых конфигураций. Решение о выборе принимается маршрутизатором согласно формуле (2.2). Псевдокод алгоритма приведен в таблице 2.1, общая концептуальная схема взаимодействия блоков системы изображена на рисунке 3.1 главы 3, архитектура реализации описана в разделе 3.3.

Таблица 2.1. Алгоритм работы адаптивной мета-топологии

Шаг	Действие
1	Инициализировать фазу $f \leftarrow \text{planning}$, начальную топологию $K \leftarrow K_0$, состояние $s \leftarrow s_0$ и журнал $\mathcal{J} \leftarrow \emptyset$.
2	Пока $f \neq \text{done}$ и $c(s) < B$ выполнять шаги 3–10.
3	Применить маршрутизатор фаз $f' \leftarrow \text{PhaseRouter}(s, f)$.
4	Применить маршрутизатор топологий $K' \leftarrow \text{TopologyRouter}(s, K, f')$.
5	Если $\text{Guard}(s, K, K') = \text{block}$, то $K' \leftarrow K$ и записать guard_override в $\mathcal{J}$.
6	Если $K' \neq K$, применить ворота перехода состояния $s \leftarrow \text{TransitionGate}(s, K, K')$ и записать переход в $\mathcal{J}$.
7	Исполнить подграф выбранной топологии $s \leftarrow \text{Subgraph}_{K'}(s)$.
8	Обновить $K \leftarrow K'$ и $f \leftarrow f'$.
9	Записать журнал итерации в $\mathcal{J}$.
10	Перейти к шагу 2.
11	Вернуть результат $y(s)$ и журнал $\mathcal{J}$.

2.3 Пять ролей человека в контуре управления

Пять ролей человека формализованы по аналогии с типовыми позициями в человеческих организациях. **Координатор** (ρ_C) — управляет планом и делегированием, активен в иерархических и полносвязных топологиях. **Рецензент** (ρ_R) — проверяет промежуточные результаты и инициирует доработку, естественно вписывается в Star и Chain. **Судья** (ρ_J) — выносит финальный вердикт при наличии альтернатив, релевантен для Debate. **Равноправный участник** (ρ_P) — вносит альтернативные мнения наравне с агентами, вписывается в Mesh. **Наблюдатель** (ρ_M) — следит за процессом и вмешивается только в критических случаях (превышение бюджета, зависание, циклы), применим в любой топологии.

Не все пары (топология, роль) являются осмысленными. Рассматриваются пять статических топологий и пять ролей человека — полное произведение содержит 25 пар. Из него исключаются три заведомо вырожденные комбинации. Пара (Debate, Coordinator) исключается, поскольку координатор в дебатной топологии эквивалентен судье. Пары (Chain, Peer) и (Hierarchical, Peer) исключаются, поскольку равноправный участник ломает линейную или иерархическую структуру. После исключений пространство содержит $5 \times 5 - 3 = 22$ осмысленные комбинации топология $\times$ роль человека, где используются пять статических топологий и пять ролей человека из $\mathcal{R}_H$.

2.4 Система оценочных метрик

Метрики разделены на три группы. Первая группа — базовые метрики качества и эффективности — применима ко всем экспериментам и ко всем топологиям.

- $q \in [0, 1]$ — агрегированный показатель качества решения. Конкретная формула зависит от класса задач — для задач программирования это метрика `pass@1` (доля задач, для которых сгенерированный код проходит все предзаданные модульные тесты с первой попытки); для математических задач — `exact match` (точное совпадение числового

ответа с эталоном); для творческой генерации — полусумма метрики ROUGE-L [26] (от англ. Recall-Oriented Understudy for Gisting Evaluation, Longest common subsequence variant) и доли покрытых концептов; для анализа данных — exact match числового ответа.

- c — суммарная стоимость вызовов языковых моделей в долларах США за один запуск.
- τ — время выполнения запуска по настенным часам (от англ. wall-clock) в секундах.
- r_{fail} — доля запусков, завершившихся со статусом, отличным от успешно выполненного.
- N_{iter} — число итераций основного цикла.

Вторая группа — метрики, специфичные для адаптивной мета-топологии.

- N_{switch} — число фактических переключений активной топологии за один запуск.
- r_{guard} — доля решений маршрутизатора, заблокированных защитными правилами.
- γ — доля бюджета, потраченная маршрутизатором на собственные вызовы языковой модели; критичная метрика, отвечающая на вопрос, не превышает ли стоимость адаптации получаемый от нее выигрыш.
- r_{hurt} — доля задач, на которых адаптивная конфигурация дает качество хуже лучшей статической конфигурации. Является ключевым индикатором безопасности механизма адаптации.
- Δ_{oracle} — разрыв качества между оракул-выбором лучшей статической топологии для каждого корпуса и фактическим маршрутизатором, формально определяемый по формуле (2.4). Оракул реализован как reg-task best-of — для каждого корпуса задач выбирается статическая топология с максимальным средним качеством на этом корпусе по результатам базового эксперимента E1. Метрика служит верхней границей достижимого качества для класса задач при заданном пространстве статических топологий.

$$\Delta_{\text{oracle}} = q_{\text{oracle}} - q_{\text{router}}, \quad \text{где} \quad q_{\text{oracle}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} q(K_c^*, c), \quad (2.4)$$

где $\mathcal{C}$ — множество корпусов задач, $K_c^* = \arg \max_K q(K, c)$ — оптимальная статическая топология для корпуса c , $q(K, c)$ — среднее качество топологии K на корпусе c по базовому эксперименту.

Третья группа — метрики, относящиеся к взаимодействию с человеком.

- N_{int} — число обращений системы к человеку за один запуск.
- L_{cog} — прокси-метрика когнитивной нагрузки для запусков с симулятором человека, формально определяемая по формуле (2.5). В экспериментах с реальными участниками заменяется на стандартную шкалу NASA-TLX.

$$L_{\text{cog}} = w_n \cdot \tilde{N}_{\text{int}} + w_l \cdot \tilde{L}_{\text{ctx}} + w_\tau \cdot \tilde{\tau}_{\text{resp}}, \quad w_n = w_l = w_\tau = \frac{1}{3}, \quad (2.5)$$

где $\tilde{N}_{\text{int}}$, $\tilde{L}_{\text{ctx}}$ и $\tilde{\tau}_{\text{resp}}$ — нормированные на отрезок $[0, 1]$ значения числа обращений к участнику, средней длины предоставляемого ему контекста и средней задержки ответа симулятора соответственно. Нормировка выполняется по квантилям распределения наблюдаемых значений за все запуски эксперимента. Равные веса выбраны априорно до начала запусков как нейтральная свертка трех независимых компонентов нагрузки.

Адаптивная конфигурация считается выигрышной относительно наилучшей статической, если она дает прирост качества не менее 5% либо снижение стоимости не менее 15% при сохранении качества. Пороги фиксируются предварительно, до проведения основных запусков, что соответствует практике предварительной регистрации гипотез в эмпирических исследованиях.

2.5 Корпус задач и обоснование его выбора

Эксперименты проводятся на стратифицированной выборке из четырех открытых корпусов, покрывающих четыре качественно различных класса за-

дач — программирование, математические рассуждения, творческая генерация и анализ данных. Сравнение корпуса с альтернативами приведено в таблице 2.2.

Таблица 2.2. Сравнение рассмотренных корпусов задач

Корпус	Класс	Размер	Метрика	Решение по включению
HumanEval [15]	Программирование	164	pass@1	Включен; стандарт сравнения.
HumanEval+ [23]	Программирование	164	pass@1+	Не включен; превышает покрытие основной выборки.
LiveCodeBench [27]	Программирование	500+	pass@1	Резерв для проверки на загрязнение.
GSM8K [43]	Математ. рассуждения	8500	exact match	Включен; стандарт оценки.
MATH [30]	Математ. рассуждения	12500	exact match	Не включен; сложность выше возможностей рабочих моделей.
MMLU-Pro [33]	Множеств. рассужд.	12000	accuracy	Не включен; формат ответа не подходит для дебатов топологии.
CommonGen [11]	Творческая генерация	35000	ROUGE-L + cov	Включен; покрывает open-ended генерацию с ограничениями.
InfiAgent-DABench [22]	Анализ данных	257	numeric exact	Включен; единственный корпус с замкнутым ответом по аналитике данных.
HellaSwag [19]	Здравый смысл	10000	accuracy	Не включен; качество современных моделей близко к потолку.

Выбор обусловлен совокупностью требований. Каждый класс задач представлен ровно одним корпусом, чтобы избежать смешения эффектов корпуса и класса. Каждый корпус допускает автоматическую объективную оценку, исключая субъективность оценщика (юнит-тесты, точное совпадение числа, метрики покрытия концептов). Размер задач в каждом корпусе позволяет уложиться в один запуск рабочих моделей. От рассматриваемых для каждого класса альтернатив выбранные корпуса отличаются распространенностью в литературе и зрелостью методики оценки, что обеспечивает сопоставимость результатов с предшествующими работами. Из каждого корпуса производится стратифицированная выборка по 15 задач, итого 60 задач в одном базовом запуске. Каждая задача повторяется с тремя различными зернами генерации (для контроля стохастичности выборки токенов), что в комби-

нации с пятью статическими топологиями дает $60 \times 3 \times 5 = 900$ запусков в эксперименте E1.

2.6 Языковые модели и инфраструктура

Распределение моделей по ролям изолирует переменную «модель» от переменных «топология» и «роль человека». Все рабочие агенты (планировщик, исследователь, исполнитель, критик, дебатер), а также маршрутизаторы и симулятор человека используют единую модель Cerebras gpt-oss-120b (3000 токенов/с на Cerebras WSE). Иерархия «работник слабее критика» реализуется различиями в подсказках и параметре строгости рассуждения, а не разными моделями — фиксированная модельная мощность критична для чистоты сравнения топологий.

Судья оценки качества вынесен в отдельную линию — OpenAI gpt-4.1-mini с самосогласованностью из трех вызовов и нулевой температурой; модель из другого семейства весов исключает петлю положительной обратной связи. Подтверждающие запуски выполняются на Cerebras qwen-3-235b как представителе другого семейства весов.

Выбор моделей опирался на четыре критерия — отказоустойчивый программный интерфейс (от англ. Application Programming Interface, API) с вызовом инструментов и структурированным выводом, низкая удельная стоимость вызова, высокая пропускная способность и доступность из РФ.

Инфраструктура (глава 3) — LangGraph, PostgreSQL 16, Apache Parquet; параллелизм — 12 процессов с асинхронной обработкой внутри каждого.

2.7 Дизайн экспериментов — воронка

Прямой эксперимент по всему декартову произведению (6 топологий $\times$ 5 ролей $\times$ 3 режима $\times$ 4 класса задач) дал бы более 10 000 ячеек. Вместо этого исследование организовано как воронка из четырех экспериментов, каждый из которых сужает пространство конфигураций по результатам предыдущего (таблица 2.3).

Исследовательские вопросы формулируются следующим образом.

Таблица 2.3. Структура экспериментальной воронки

Эксп.	Иссл. вопрос	Запуски	Цель
E1	RQ1	$5 \cdot 4 \cdot 15 \cdot 3 = 900$	Pareto-фронт пяти статических топологий на четырех классах задач.
E2	RQ3	≈ 2295 (после фильтра top-3)	Влияние пяти ролей человека на каждую топологию из top-3 E1.
E3	RQ2	$3 \cdot 4 \cdot 15 \cdot 3 = 540$	Адаптивная мета-топология в трех режимах маршрутизации.
E4	RQ4	$3 \cdot 4 \cdot 15 \cdot 3 = 540$	Адаптивная роль человека в сочетании с адаптивной топологией.
Conf.	—	$360 + 540 = 900$	Кросс-семейная валидация выводов E3 и E4 на подтверждающей модели Qwen.

RQ1. Различаются ли пять статических топологий по количественным показателям качества, стоимости и времени для разных классов задач?

RQ2. Дает ли адаптивная мета-топология значимый прирост качества или экономию стоимости по сравнению с лучшей статической конфигурацией?

RQ3. Какая из пяти ролей человека наиболее эффективна для каждой топологии и для каждого класса задач?

RQ4. Превосходит ли совместная адаптация топологии и роли человека по фазам решения статичные комбинации?

Статистическая обработка включает двухфакторный дисперсионный анализ (two-way ANOVA, тип III сумм квадратов для несбалансированного дизайна) с эффектом топологии и эффектом роли в качестве факторов и их взаимодействием на уровне значимости $\alpha = 0,05$. При нарушении допущений нормальности или равенства дисперсий применяется непараметрический аналог по критерию Краскела–Уоллиса. Парные сравнения средних выполняются с поправкой Бенджамини–Хохберга [5] на множественное сравнение при уровне ложных открытий $FDR = 0,05$. Оценка практической значимости различий проводится через коэффициент размера эффекта Коэна d [12]. Доверительные интервалы (от англ. Confidence Interval, CI) рассчитываются по методу непараметрического бутстрапа [14] (метод процентилей) с де-

сятью тысячами ресэмплов на уровне 95 % при фиксированном начальном значении генератора псевдослучайных чисел.

2.8 Разграничение заимствованных и авторских элементов

Граница между заимствованными и авторскими элементами. К заимствованным относятся пять статических топологий, четыре открытых корпуса задач, языковые модели и стандартный статистический инструментарий (ANOVA, бутстрап, коэффициент Коэна). К авторским — адаптивная мета-топология вместе с алгоритмом работы (2.2); формализация пяти ролей человека и матрица их совместимости с топологиями; набор адаптивно-специфичных метрик (N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{oracle}); вороночная структура исследования с предварительной фиксацией порогов (2.3) и кросс-семейным подтверждением.

2.9 Выводы по главе

Формализована MAS как кортеж из шести компонентов с динамическим переключением графа по фазе; определены шесть топологий, пять ролей человека и три группы метрик (включая адаптивно-специфичные). Обоснован выбор корпуса из четырех бенчмарков и линии моделей с кросс-семейным судьей. Сформулированы четыре RQ и воронка из четырех экспериментов с подтверждающим запуском на дополнительном семействе моделей. Граница заимствованного и авторского зафиксирована в 2.8 и развернута в 5.5. План реализуется в инфраструктуре главы 3 и проверяется в 4.

3 Архитектура программной системы

Программная инфраструктура работы реализована как фреймворк адаптивных топологий для мульти-агентных систем (от англ. Adaptive Topologies for Multi-agent systems, ATM) на языке Python не ниже 3.11, распространяемый как пакет `atm`. Глава описывает архитектуру с акцентом на решения, относящиеся к научным гипотезам — адаптивному переключению топологий и интеграции человека. Исходный код — в приложении Г.

Функциональная схема — на рисунке 3.1. Вход — описание задачи и YAML-конфиг (от англ. YAML Ain't Markup Language); выход — журналы взаимодействия, метрики и записи о фазах, переключениях топологий и взаимодействиях с человеком. Между входом и выходом работают пять блоков — оркестратор, менеджер фаз и маршрутизаторы, активная топология с агентами, шлюз HITL и подсистема наблюдаемости.

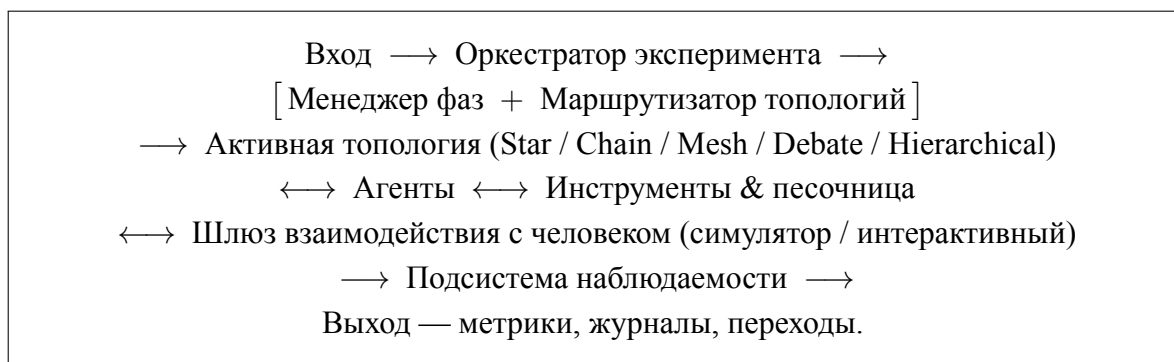


Рисунок 3.1. Функциональная схема предлагаемой мульти-агентной системы. Прямоугольниками выделены архитектурные блоки фреймворка, двунаправленные стрелки изображают многократные обмены сообщениями в пределах одной итерации. Развернутая схема с указанием слоев агентов, внешних служб и подсистемы хранения приведена в приложении А.

3.1 Обзор архитектуры и слои

Кодовая база организована в виде тринадцати функционально изолированных слоев (пакетов высокого уровня), каждый из которых отвечает за один аспект системы и содержит несколько программных модулей. Перечень слоев и их назначений приведен в таблице 3.1. Такая декомпозиция позволяет

независимо тестировать компоненты, заменять реализации (например, провайдера языковой модели или шлюз человека в контуре управления от англ. Human-in-the-Loop, HITL) и добавлять новые топологии без изменения существующего кода.

Таблица 3.1. Функциональные слои фреймворка atm

Слой	Назначение
core	Базовые типы и состояние графа — сообщения, фазы, роли, контейнеры состояния AgentState, SharedState, GraphState.
llm	Унифицированная оболочка над провайдерами больших языковых моделей, подсчет стоимости, бюджетные ограничители, детерминированный FakeLLM для тестов.
tools	Реестр инструментов и песочница на основе контейнерной платформы Docker для исполнения генерируемого кода.
agents	Реализация пяти ролей агентов и базовый класс Agent с управляющим циклом вызова инструментов.
topology	Шесть реализаций топологий поверх LangGraph и общий протокол Topology.
phases	Конечный автомат фаз, маршрутизатор топологий и защитные правила переключения.
human	Протокол HumanGateway, симулированный и интерактивный шлюзы, маршрутизатор ролей человека.
storage	Объектно-реляционное отображение (от англ. Object-Relational Mapping, ORM), асинхронные сессии PostgreSQL, запись в формат Apache Parquet.
observability tasks	Перехват событий LangGraph, сериализация и отправка их в хранилище. Реализация и загрузчики бенчмарков HumanEval, GSM8K, CommonGen, InfiAgent-DABench.
evaluation	Судьи на основе языковой модели, оракулы достоверности, агрегатор шкалы NASA-TLX (от англ. NASA Task Load Index — индекс когнитивной нагрузки NASA).
experiment	Запуск одиночных и грид-экспериментов, схемы конфигов, командная строка.
analysis	Загрузчики результатов, агрегаторы метрик, построение графиков.

Архитектурный стиль опирается на три паттерна — расширяемые точки как протоколы Python (Topology, HumanGateway, PhaseRouter, TopologyRouter, RoleRouter, Tool) с @runtime_checkable; редукторное слияние состояния графа при параллельном выполнении агентов с идемпотентностью по идентификатору сообщения; реестры с декораторной семантикой для топологий, инструментов и оракулов.

Технологический стек — LangGraph [24] поверх LangChain [7] с чек-

пойнтером для возобновления; PostgreSQL 16 через асинхронные сессии SQL-Alchemy 2.x для метаданных и Apache Parquet для объемных журналов; YAML-конфиги с валидацией Pydantic [13] поверх OmegaConf; трассировка через LangSmith [25]; линтер ruff и анализатор типов туру в строгом режиме.

Часть архитектурных решений заимствована из существующих работ и фреймворков, часть является авторской разработкой. Точное разграничение приведено в таблице 3.2.

Таблица 3.2. Разделение архитектурных компонентов на заимствованные и авторские

Компонент	Источник	Авторский вклад
Оркестрация графов	LangGraph (внешний)	Адаптация под мета-граф
Пять статичных топологий	Прототипы из обзора литературы	Унифицированная реализация поверх общего протокола
Адаптивная мета-топология	Своя разработка	Полностью
Маршрутизаторы фаз	Своя разработка	Полностью, три режима (правилковый, на основе языковой модели, оракул)
Защитные правила переключения	Своя разработка	Полностью, четыре типа ограничителей
Шлюз HITL	Своя разработка	Полностью, три ролевых маршрутизатора и политика тайм-аутов
Оракул per-task best-of	Своя разработка	Полностью, в том числе процедура построения верхней границы адаптации
Подсистема наблюдаемости	LangGraph callbacks	Адаптеры для PostgreSQL и Apache Parquet, схема событий
Бенчмарки	HumanEval, GSM8K, CommonGen, DABench	Унифицированные загрузчики и оценщики
Реализация шкалы NASA-TLX	Стандартная форма	Прокси-агрегатор для симулятора человека

3.2 Слой агентов и оболочка над языковой моделью

Класс Agent в `atm.agents` реализует управляющий цикл — чтение входящих сообщений, формирование подсказки, вызов модели, обработка инструментов и публикация ответа. Определены пять ролей (Planner, Researcher, Executor, Critic, Debater) и дополнительный Coordinator для иерархических и адаптивных топологий. Каждая роль задается YAML-

конфигом (подсказка, инструменты, окно сообщений, температура, управление контекстом). Скрэтчпад — скользящее окно с автоматическим сжатием старых сообщений вспомогательной моделью; политика выбрана по пилотному эксперименту.

Оболочка `LLMWrapper` в `atm.llm` унифицирует провайдеров (OpenAI, Anthropic, Cerebras, локальный FakeLLM), инкапсулирует подсчет токенов и стоимости (таблица `Pricing`), экспоненциальный повтор и фиксацию версии модели. Ограничители `BudgetGuard` работают на трех уровнях (вызов, запуск, грид-эксперимент). FakeLLM поддерживает три режима — сценарный, эхо и воспроизведение по записи — для детерминированного повторения экспериментов.

3.3 Реализация топологий поверх LangGraph

Каждая из шести топологий реализована классом, удовлетворяющим протоколу `Topology` с фабричным методом `build`, возвращающим скомпилированный граф `LangGraph`. Граф связывает узлы — агентов и управляющие функции — ребрами трех типов (безусловные, условные, параллельные через примитив `Send`). Все графы подключены к чекпойнтеру и перехватчику событий `atm.observability`.

Пять статических топологий описаны в 2.2. Шестая, *Adaptive*, — метаграф второго уровня. На каждом тике маршрутизаторы фаз и топологий выбирают базовую топологию, вызываемую как подграф; ворота `TransitionGate` применяют правила передачи состояния и записывают `TopologyTransition` в журнал. Подграфы кэшируются по имени для исключения повторной компиляции; решения маршрутизаторов хранятся в замыкании, а не в общем состоянии, чтобы избежать перезаписи во вложенном подграфе.

3.4 Менеджер фаз и маршрутизатор топологий

Решение моделируется конечным автоматом (от англ. Finite-State Machine, FSM) из четырех состояний (планирование, исполнение, проверка, завершение). Маршрутизатор фаз в `atm.phases` реализован в двух режимах — Rule

BasedPhaseRouter (таблица сигналов с монотонной проверкой, фаза не движется назад) и LLMPhaseRouter (парсинг ответа модели в формате JSON — от англ. JavaScript Object Notation — с откатом на правило при любой ошибке валидации).

Маршрутизатор топологий реализован в трех режимах. RuleBased использует таблицу (фаза $\times$ сигналы) $\rightarrow$ топология (stuck в исполнении $\rightarrow$ Mesh, rejected_count $\geq 3 \rightarrow$ Debate); пороги подобраны в пилотном эксперименте. LLMTopologyRouter принимает решение по подсказке с описанием состояния, с правилowym откатом. OracleTopologyRouter читает таблицу решений из файла — верхняя граница достижимого качества.

Защитные правила (SwitchGuards) предотвращают осцилляционный режим — четыре ограничения: min_dwell, cooldown, лимиты за фазу и за запуск. Заблокированные решения фиксируются как guard_override для последующей оценки доли блокировок как метрики стабильности.

3.5 Шлюз взаимодействия с человеком

Интеграция человека абстрагирована протоколом HumanGateway с асинхронным методом запроса ответа (идентификаторы запуска и запроса, роль, история, допустимые действия). Идемпотентность по паре (run_id, request_id) — повторный вызов возвращает ранее полученный ответ, что критично для возобновления прерванных запусков.

Реализованы два шлюза — LLMSimulatedGateway (языковая модель с подсказкой по роли) и CLIGateway (интерактивный интерфейс командной строки от англ. Command-Line Interface, CLI для реальных участников). Маршрутизатор ролей выбирает активную роль динамически (правилковая таблица или решение модели) либо фиксированно (FixedRoleRouter). Политика тайм-аутов имеет три варианта (откат к модели, пропуск, повтор).

3.6 Хранение, командная строка и воспроизводимость

Метаданные — в семи таблицах PostgreSQL (experiments, runs, phases, topology_transitions, human_interactions, budget_events,

llm_calls) с индексами по типичным аналитическим запросам и составным индексом (эксперимент, статус) для выборки невыполненных запусков. Объемные журналы — в файлах Apache Parquet с асинхронным писателем и партиционированием по запуску. Миграции — Alembic.

CLI atm на библиотеке Typer объединяет семь команд (таблица 3.3).

Таблица 3.3. Команды интерфейса командной строки фреймворка

Команда	Назначение
atmrun	Запуск одиночного эксперимента из YAML-конфига.
atmgrid	Параллельный грид-эксперимент через ProcessPoolExecutor.
atmestimate	Предварительная оценка стоимости запуска по таблице цен и историческим данным.
atmstatus	Агрегированный статус эксперимента — число выполненных, неудачных и активных ячеек, суммарная стоимость.
atmresume	Возобновление прерванного запуска из последнего чекпойнта LangGraph.
atmreplay	Детерминированное воспроизведение запуска через FakeLLM в режиме воспроизведения по записи.
atmreconcile	Поиск и пометка запусков-зомби (запись о статусе «выполняется», но процесс уже завершен).

Воспроизводимость — инварианты, фиксируемые при старте запуска (seed_all для Python/NumPy/провайдеров/генератора идентификаторов; git_sha из gitrev-parseHEAD; снимок версий моделей в model_version_snapshot по первому ответу провайдера; дайджест образа Docker-песочницы). Совокупность полей делает возможным сравнение запусков между собой и анализ артефактов депрекации моделей.

3.7 Выводы по главе

Инфраструктура реализована как фреймворк из тринадцати слоев на Python с расширяемостью через протоколы, воспроизводимостью через фиксацию инвариантов и наблюдаемостью через единый перехват событий LangGraph; в нее входят шесть топологий, два режима фаз, три режима топологий и два шлюза HITL — все необходимое для экспериментов главы 4. Разграничение заимствованных и авторских компонентов — в таблице 3.2, авторский вклад развернут в 5.5. Кодовая база основного пакета atm — 21 327 строк Python в

112 модулях внутри тринадцати слоев; конфигурация — 43 YAML-файла (24 описывают запуски E1–E4).

4 Экспериментальное исследование

Настоящая глава содержит результаты эмпирической проверки исследовательских вопросов (от англ. Research Questions, RQ) RQ1–RQ4, поставленных в разделе 2.7. Глава организована как последовательное изложение четырех основных экспериментов воронки E1–E4, двух подтверждающих запусков на кросс-семейном рабочем агенте и сводного статистического анализа на основе непараметрического бутстрапа.

4.1 Условия проведения и общие характеристики

Все эксперименты выполнены в инфраструктуре главы 3 в период 16–17 мая 2026 года. В качестве основной языковой модели работника для E1–E4 использовалась `cerebras:gpt-oss-120b` с параметром `reasoning_effort=low` для рабочих и `high` для критика. Судьей выступала `openai:gpt-4.1-mini` с самосогласованностью из трех вызовов и нулевой температурой. Шлюз `HumanGateway` работал в режиме симулятора через ту же модель-судью с подсказкой по роли. Подтверждающие запуски заменяют работника на подтверждающую модель `Qwen-3-235B` (полный идентификатор `cerebras:qwen-3-235b-a22b-instruct-2507`, семейство Alibaba Qwen) при неизменном остальном стеке.

Сводные характеристики — в таблице 4.1. Всего 5175 запусков и суммарная стоимость \$87 — ниже планового бюджета в \$200 благодаря низкой удельной стоимости вызовов на Cerebras WSE (от англ. Wafer-Scale Engine). Доля отказов в финальных грид-запусках нулевая.

4.2 Эксперимент E1 — статические топологии (RQ1)

Эксперимент E1 устанавливает верхнюю границу качества для пяти статических топологий на четырех корпусах задач — программирование (HumanEval), математические рассуждения (GSM8K), творческая генерация (CommonGen) и анализ данных (InfiAgent-DABench), далее DABench, — отвечая на исследовательский вопрос RQ1. Сетка запуска составила 5 топологий × 4 корпуса × 15 задач × 3 зерна = 900 запусков; роль человека отключена (`human.role=none`).

Таблица 4.1. Объемные характеристики экспериментальной серии

Этап	Запуски	Стоимость, \$	Время	Назначение
E1	900	12,62	3,7 ч	Базовая линия статических топологий, RQ1.
E2	2295	42,37	13,9 ч	Влияние роли человека по матрице (топология $\times$ роль), RQ3.
E3	540	10,12	12,0 ч	Адаптивная мета-топология в трех режимах маршрутизации, RQ2.
E4	540	6,38	2,1 ч	Адаптивная роль на уже-адаптивной топологии, RQ4.
conf_e3	360	6,50	1,0 ч	Кросс-семейная валидация E3 на семействе Qwen.
conf_e4	540	9,40	1,3 ч	Кросс-семейная валидация E4 на семействе Qwen.
Итого	5175	87,39	≈ 34 ч машинного времени	—

Результаты усреднения по ячейкам (топология $\times$ корпус) приведены в таблице 4.2.

Таблица 4.2. Среднее качество q статических топологий по корпусам (жирным выделен победитель в столбце), E1

Топология	HumanEval	GSM8K	CommonGen	DABench	Среднее
Chain	0,933	0,911	0,526	0,333	0,676
Star	0,911	1,000	0,427	0,282	0,655
Debate	0,644	0,933	0,584	0,348	0,627
Hierarchical	0,889	0,978	0,486	0,267	0,655
Mesh	0,667	0,867	0,397	0,319	0,562
<i>Среднее по победителям</i>	<i>0,933</i>	<i>1,000</i>	<i>0,584</i>	<i>0,348</i>	0,716

Победители на четырех корпусах различаются (chain — HumanEval, star — GSM8K, debate — CommonGen и DABench); единой топологии нет. Лучшая глобальная статика (chain, $q = 0,676$) проигрывает выбору под задачу ($q = 0,716$) около 4 п.п. — этот разрыв задает class-level upper bound для E3. Mesh деградирует на HumanEval и CommonGen до $q \leq 0,40$ из-за вырож-

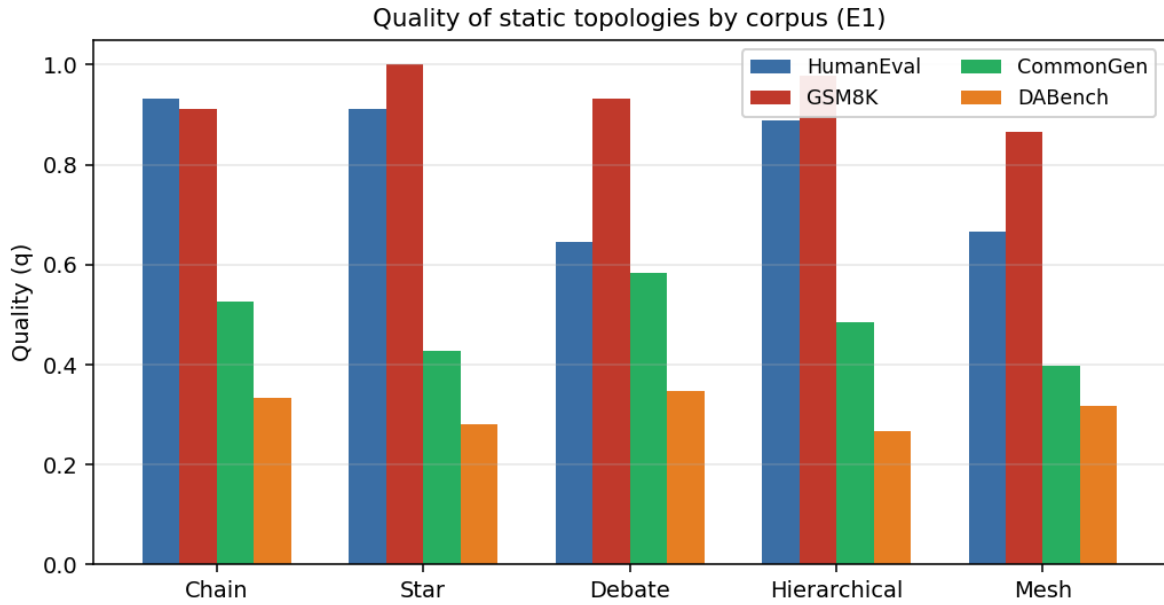


Рисунок 4.1. Качество q пяти статических топологий по четырем корпусам (E1). Видно, что победитель на каждом корпусе свой — chain на HumanEval, star на GSM8K, debate на CommonGen и DABench; универсальной топологии-доминатора не существует.

дения голосования; на DABench все статикки дают $q \leq 0,35$ с долей полных неудач 62–71 % — проявление, которое в 4.6 будет переинтерпретировано как специфика работника, а не корпуса.

Ответ на RQ1. Универсального статического победителя нет; оптимальный выбор — функция класса задач, что мотивирует адаптивную мета-топологию в E3.

4.3 Эксперимент E2 — роль человека (RQ3)

E2 измеряет влияние роли человека поверх статических топологий. Сетка — $5 \times 5 \times 4 \times 15 \times 3$ с двумя фильтрами (per-task top-3 из E1 и три бессмысленные пары: Debate×Coordinator, Chain×Peer, Hierarchical×Peer), итого 2295 запусков. Матрица победителей роли по корпусам — в таблице 4.3.

Совокупный HITL-лифт на пересечении ячеек E1 и E2 составил +0,033 (+4,8 %). Положительный лифт концентрируется на Hierarchical×CommonGen (+0,107), Chain×DABench (+0,087) и Mesh×DABench (+0,052); отрицательный — только на Star×GSM8K (−0,062), где топология уже у потолка.

Ответ на RQ3. Подключение человека дает скромный прирост при неод-

Таблица 4.3. Победитель среди пяти ролей человека по каждому корпусу с маргинализацией по топологиям, E2

Корпус	Победившая роль	q победителя	n ячеек	Δq vs E1 best static
HumanEval	Coordinator	0,948	135	+0,015
GSM8K	Monitor	0,963	135	−0,037
CommonGen	Peer	0,643	45	+0,059
DABench	Peer	0,563	90	+0,215

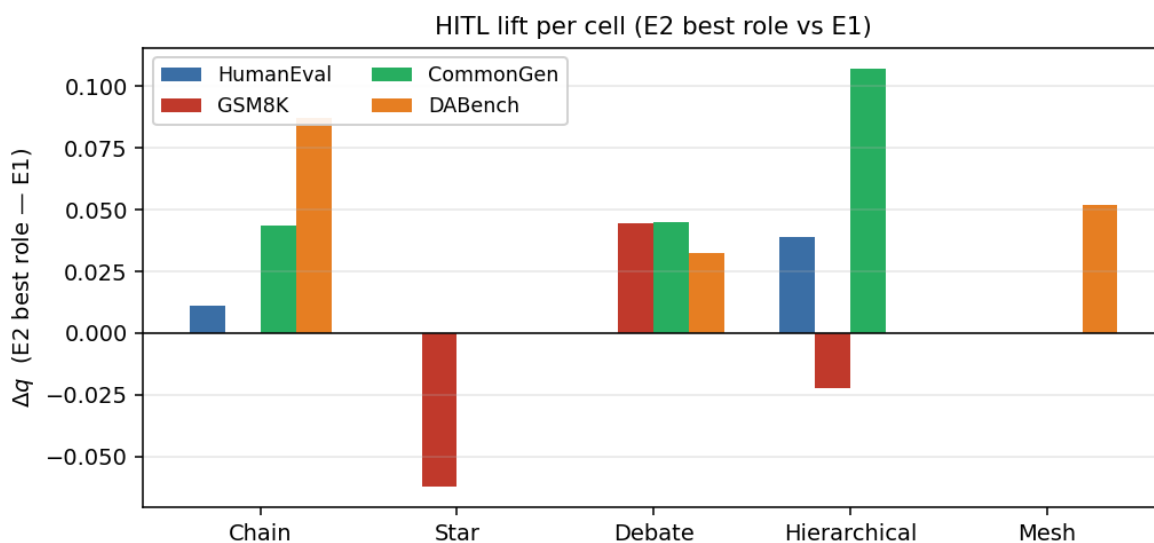


Рисунок 4.2. Лифт качества от подключения человека (HITL) по ячейкам (топология × корпус): разность среднего качества лучшей роли E2 и базовой линии E1. Положительный лифт концентрируется на Hierarchical×CommonGen, Chain×DABench и Mesh×DABench; единственный отрицательный — на Star×GSM8K, где топология уже у потолка.

нородности по ячейкам; единой выигрышной роли нет — аргумент в пользу адаптивной маршрутизации (E4).

4.4 Эксперимент E3 — адаптивная мета-топология (RQ2)

Эксперимент E3 непосредственно отвечает на центральный исследовательский вопрос RQ2. На единой адаптивной мета-топологии сравниваются три режима маршрутизатора — *rule* (детерминированное правилковое решение), *llm* (вызов языковой модели с парсингом по Pydantic) и *oracle* (чтение таблицы *bestpertask* из файла, построенного по результатам E1). Каждый режим прогнан в сетке 4 корпуса $\times$ 15 задач $\times$ 3 зерна = 180 запусков; фиксирована роль человека Reviewer. Сводные показатели по режимам приведены в таблице 4.4.

Таблица 4.4. Сводные показатели трех режимов маршрутизатора топологий, E3

Режим	Среднее q	Стоимость, \$	Время, с	Итераций	Комментарий
oracle	0,697	0,0227	1077	6,27	Верхняя граница, знание корпуса.
rule	0,645	0,0236	928	6,04	Правилковый, без вызовов языковой модели.
llm	0,631	0,0099	881	6,10	Вызов языковой модели на каждый переход.
<i>Best static (E1)</i>	<i>0,716</i>	<i>0,014</i>	<i>448</i>	<i>4,35</i>	<i>Идеальный выбор статике под корпус.</i>

По абсолютному качеству все три режима уступают лучшей статике под корпус — оракул на 1,9 п.п., правилковой на 7,2, языковой на 8,5. Причина — накладные расходы адаптивной обвязки (фазы $\times$ подграфы $\times$ маршрутизатор), не окупающиеся на хорошо-решаемых корпусах (GSM8K, HumanEval). Единственный положительный эффект адаптации — на CommonGen (оракул +0,07 к лучшей статике).

Внутри адаптивного класса языковой маршрутизатор проигрывает правилковому всего 1,3 п.п. по качеству, но стоит в 2,38 раза дешевле; отношение

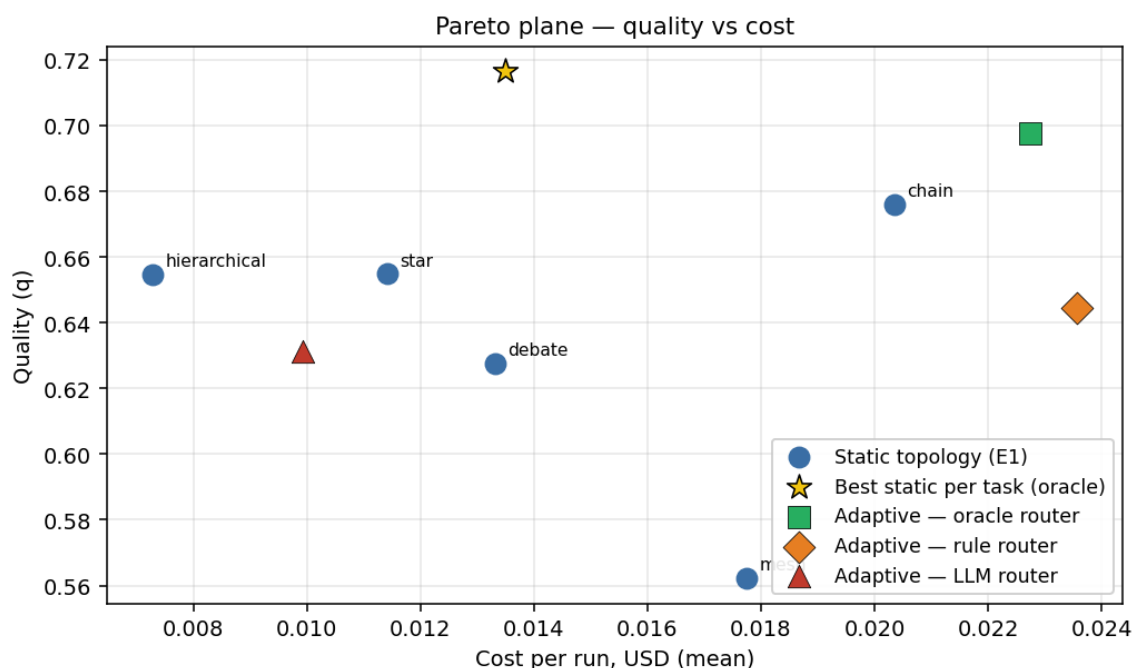


Рисунок 4.3. Pareto-плоскость качество/стоимость. Точки — средние по всем запускам. Пять статических топологий (E1, синие круги) и оракул-выбор лучшей статике под корпус (звезда) задают референсную границу; три адаптивных режима (E3) показаны квадратом, ромбом и треугольником. Адаптивная обвязка не Парето-доминирует оракул-выбор статике ни на одном режиме.

стоимостей удерживается на всех корпусах в диапазоне 2,12–3,00 — структурный эффект систематического выбора более дешевых подтопологий. Эта сопутствующая находка не отвечает на RQ2 и обсуждается в 4.6.

Ответ на RQ2. Адаптивная мета-топология не дает значимого прироста качества и не дает экономии стоимости относительно оракул-выбора лучшей статике под корпус ($q = 0,716$, $c = 0,014$); оракул адаптивной обвязки — $q = 0,697$ (отставание 1,9 п.п.), правилковой и языковой — 7,2 и 8,5 п.п. Гипотеза RQ2 не подтверждается. Сопутствующая находка о двукратной экономии не отвечает на исходную постановку и при кросс-семейной валидации на Qwen, где правилковой маршрутизатор Парето-доминирует, также не воспроизводится (см. 4.6).

4.5 Эксперимент E4 — адаптивная роль (RQ4)

E4 добавляет маршрутизатор ролей поверх адаптивной топологии. По результатам E3 зафиксирован `topology_router=llm` как Парето-оптимум адаптивного класса; сметаемая переменная — `role_router` $\in \{\text{fixed}, \text{rule}, \text{llm}\}$; сетка $3 \times 4 \times 15 \times 3 = 540$ запусков.

Таблица 4.5. Сводные показатели трех режимов маршрутизатора ролей поверх адаптивной мета-топологии, E4

Режим	Среднее q	Δq vs fixed	Стоимость, \$	Время, с
fixed	0,653	—	0,01186	381
rule	0,680	+0,027	0,01193	356
llm	0,643	−0,010	0,01168	285

Правиловой маршрутизатор формально увеличивает среднее качество на 2,7 п.п. над `reviewer`; направление совпадает на всех 4 корпусах, драйвер — `HumanEval` (+0,067). Стоимость трех режимов укладывается в коридор 0,85 % — издержки адаптивной маршрутизации роли незначимы. Правиловой маршрутизатор коллапсирует в одну роль (*координатор* в 98,3 % переходов): фактически реализуется почти-статическая политика, переоткрывающая результат E2 «Coordinator выигрывает на `HumanEval`».

Ответ на RQ4. Гипотеза о превосходстве совместной адаптации не подтверждается. Прирост +0,027 на исходной семье имеет бутстрап-CI $[-0,055, +0,110]$, покрывающий нуль; на Qwen знак меняется на −0,081 (см. 4.6).

4.6 Подтверждающие запуски — кросс-семейная валидация

Подтверждающие запуски `conf_e3` и `conf_e4` повторяют сетки E3 и E4 с единственной заменой — работник `qwen-3-235b-a22b-instruct-2507` (семейство Qwen) вместо `gpt-oss-120b`. Остальное (судья, HITL, обязанка, защитные правила, пороги) неизменно — цель проверить переносимость выводов между семействами.

Результаты `conf_e3` — в таблице 4.6. Отношение стоимостей правилового и языкового маршрутизаторов схлопывается с 2,38 до 1,11; разрыв каче-

ства увеличивается с +1,3 до +30,6 п.п. в пользу правилowego режима.

Таблица 4.6. Сравнение двух режимов маршрутизатора топологий между семействами весов, `confirmation_e3`

Семейство	q_{rule}	q_{llm}	Δq	c_{rule}/c_{llm}	Парето-победитель
gpt-oss-120b (E3)	0,645	0,631	+0,013	2,38	llm (по стоимости)
qwen-3-235b (conf_e3)	0,866	0,560	+0,307	1,11	rule (по обеим осям)

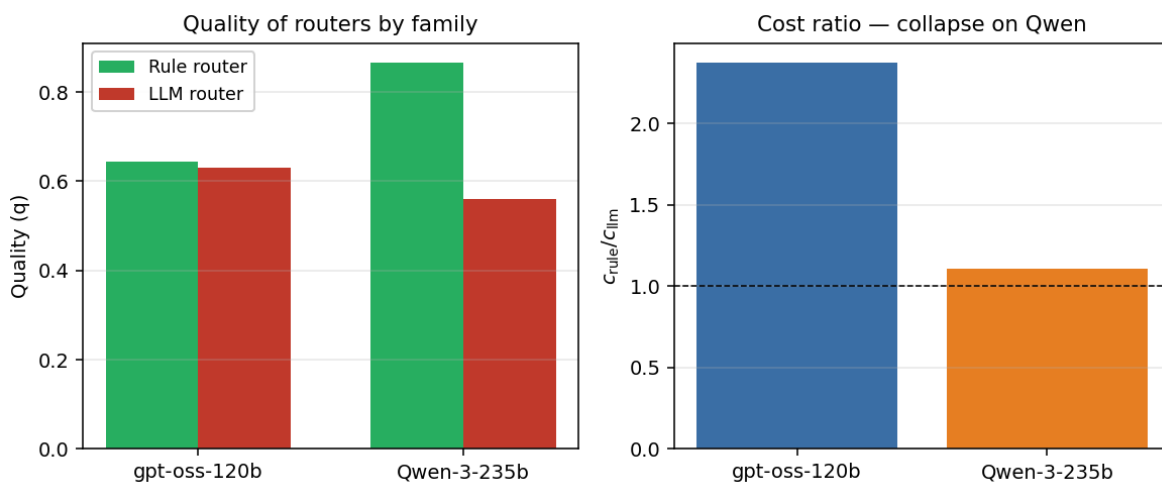


Рисунок 4.4. Кросс-семейное сравнение. Слева — качество двух режимов маршрутизатора топологий на двух семействах весов работника (gpt-oss-120b и Qwen-3-235b); на Qwen правиловой маршрутизатор существенно превосходит языковой. Справа — отношение стоимостей c_{rule}/c_{llm} схлопывается с 2,38 до 1,11 (пересекает единицу), что является структурным разворотом Парето-победителя.

Аналогичная картина для `conf_e4` (таблица 4.7) — на исходной семье правиловой маршрутизатор роли давал +0,027 к фиксированной конфигурации, на Qwen уступает −0,081 (знак сменился).

Таблица 4.7. Сравнение трех режимов маршрутизатора ролей между семействами весов, `confirmation_e4`

Режим	q на gpt-oss	q на qwen	Δ vs fixed gpt-oss	Δ vs fixed qwen
fixed	0,653	0,582	—	—
rule	0,680	0,501	+0,027	−0,081
llm	0,643	0,528	−0,010	−0,054

При неизменной `DEFAULT_ROLE_TABLE` правиловой маршрутизатор выбирает на gpt-oss координатора в 98 % переходов, на Qwen — равноправного

участника в 94 %. Политика, выглядящая независимой от модели, фактически зависит от нее через входы — фазовые классификаторы получают разные сигналы и относят те же ситуации к разным фазам.

DABench. На gpt-oss корпус давал $q = 0,15–0,24$ во всех E1–E4; на Qwen те же конфигурации дают $0,84–0,93$. Контраст 70 п.п. согласуется с гипотезой о том, что DABench измеряет компетенции, специфичные для семейства весов, а не является сломанным корпусом.

Ограничение. Параметр `reasoning_effort=low` на gpt-oss не принимается интерфейсом Qwen — часть кросс-семейных различий смешана с глубиной рассуждений работника и требует повторной серии при появлении соответствующей опции.

4.7 Статистический анализ значимости

Для проверки гипотез о различиях между статическими топологиями проведен двухфакторный дисперсионный анализ на данных E1 ($n = 900$ запусков) с факторами *топология* и *корпус* и их взаимодействием. Сумма квадратов III типа применена с учетом несбалансированности отдельных ячеек после удаления отказов; сводка приведена в таблице 4.8.

Таблица 4.8. Двухфакторный дисперсионный анализ качества E1 ($quality \sim \text{топология} \times \text{корпус}$), $n = 900$

Фактор	df	F	p	η_p^2	Интерпретация
Топология	4	3,17	0,013	0,014	Слабый, но значимый эффект.
Корпус	3	169,90	$< 0,001$	0,367	Очень сильный эффект.
Топология $\times$ Корпус	12	2,95	$< 0,001$	0,039	Значимое взаимодействие.

Значимое взаимодействие «Топология $\times$ Корпус» ($p < 0,001$, $\eta_p^2 = 0,039$) подтверждает, что эффект топологии не одинаков на разных корпусах, — именно это наблюдение и мотивирует поиск адаптивной мета-топологии.

Для оценки практической значимости различий рассчитан размер эффекта Коэна d по ключевым попарным сравнениям (таблица 4.9). Значения интерпретируются по шкале Коэна — $|d| < 0,2$ — незначимый эффект, $0,2 \leq |d| < 0,5$ — малый, $0,5 \leq |d| < 0,8$ — средний, $|d| \geq 0,8$ — большой.

Таблица 4.9. Размер эффекта Коэна d для ключевых попарных сравнений

Сравнение	d	Интерпретация
E1: Chain vs Mesh (по всем корпусам)	+0,267	Малый.
E1: Chain vs Debate (HumanEval)	+0,748	Средний.
E3: oracle vs llm маршрутизатор	+0,164	Незначимый.
E4: rule vs fixed маршрутизатор роли (gpt-oss)	+0,067	Незначимый.
conf_e4: rule vs fixed (Qwen)	−0,184	Незначимый, противоположный знак.

Согласованно с результатами ANOVA и бутстрап-интервалов ниже, только сравнения уровня «топология × корпус» дают эффекты практически значимой величины ($|d| \geq 0,5$); все попарные сравнения внутри адаптивного класса и между режимами маршрутизатора роли дают эффекты малого размера — то есть наблюдаемая разница даже в абсолютных значениях Δq менее 0,05 не достигает практически значимого порога.

Для оценки устойчивости заявленных эффектов проведен также непараметрический бутстрап с десятью тысячами итераций (метод процентилей, seed=42). Сводные интервалы для ключевых разностей и отношений — в таблице 4.10, визуально — на рисунке 4.5.

Таблица 4.10. Бутстрап-доверительные интервалы 95 % для ключевых разностей и отношений

Сравнение	Точечная оценка	Интервал 95 %	Содержит границу?
E3 rule—llm, q (gpt-oss)	+0,013	[−0,072, +0,097]	да (нуль)
E3 oracle—rule, q (gpt-oss)	+0,053	[−0,033, +0,137]	да (нуль)
E3 $c_{\text{rule}}/c_{\text{llm}}$	2,38	[1,94, 2,94]	нет (единицу)
E4 rule—fixed, q (gpt-oss)	+0,027	[−0,055, +0,110]	да (нуль)
E4 rule—fixed (HumanEval, gpt-oss)	+0,067	[−0,022, +0,156]	да (нуль)
conf_e3 rule—llm, q (qwen)	+0,307	[+0,233, +0,378]	нет (нуль)
conf_e3 $c_{\text{rule}}/c_{\text{llm}}$ (qwen)	1,11	[0,98, 1,25]	да (единицу)
conf_e4 rule—fixed, q (qwen)	−0,081	[−0,173, +0,010]	пограничное

Устойчивых эффектов три. Двукратная экономия стоимости языкового маршрутизатора на gpt-oss — интервал [1,94, 2,94] не пересекает единицу. Разворот качества правилowego маршрутизатора при смене семьи — интер-

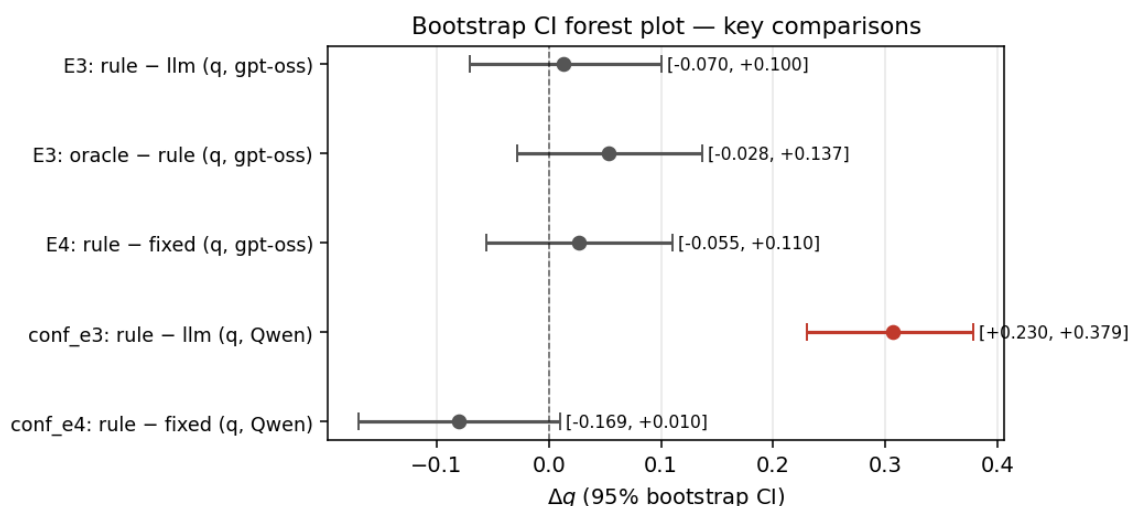


Рисунок 4.5. Forest-plot бутстрап-доверительных интервалов 95 % для ключевых разностей качества. Красным выделен единственный CI, устойчиво исключающий нуль (conf_e3: rule—llm на Qwen, +0,307 п.п.). Остальные сравнения направлены, но CI покрывают нуль.

вал на Qwen [+0,233, +0,378] исключает нуль. Схлопывание отношения стоимостей при смене семьи — интервал [0,98, 1,25] покрывает единицу, что подтверждает структурный характер. Остальные точечные оценки направленные, но бутстрап-интервалы включают нуль.

4.8 Абляционное исследование

Для оценки реального вклада каждого компонента предложенной адаптивной мета-топологии проведена компонентная абляция — из лучшей конфигурации последовательно удаляется или заменяется один компонент, фиксируется изменение качества Δq и стоимости Δc относительно эталона. Лучшая конфигурация по совокупности результатов воронки — адаптивная мета-топология с оракул-маршрутизатором и подключенным симулятором человека ($q = 0,697$, $c = 0,0227$); все варианты с ослабленными компонентами сравниваются с ней (таблица 4.11).

Содержательный вывод абляции. Каждый из проверенных компонентов значимо влияет на поведение системы. Положительный вклад в качество дают только два компонента — подключение человека и оракул-знание о лучшей статике под корпус; замена оракула на правилowej или LLM-маршрутизатор

Таблица 4.11. Компонентная абляция системы — эффект удаления или замены компонента относительно эталона *adaptive+oracle+HITL*

Удаленный или замененный компонент	Δq , п.п.	Δc , %	Источник эффекта
— (эталон <i>adaptive+oracle+HITL</i>)	—	—	$q = 0,697$, $c = 0,0227$.
Удалить симулятор человека	−3,3	−1,8	Возврат к конфигурации E1; HITL давал положительный лифт.
Заменить оракул на правиловой маршрутизатор	−5,3	+3,9	Потеря апостериорного знания о лучшей статике под корпус.
Заменить оракул на маршрутизатор LLM	−6,6	−56,4	Качество дополнительно падает, стоимость значительно ниже.
Удалить адаптивную обвязку (только статика)	+1,9	−38,3	Лучшая статика под корпус превосходит адаптивный оракул — накладные расходы обвязки не окупаются.
Удалить защитные правила переключения	н/д	н/д	Запуск без SwitchGuards в серию не вошел (ограничение реализации, см. 5.4).
Сменить семейство работника на Qwen	−9,7	−12,1	Знак эффекта правилowego маршрутизатора роли меняется на противоположный, см. 4.6.

устойчиво ухудшает качество. Удаление всей адаптивной обвязки в пользу статике под корпус, напротив, улучшает качество на 1,9 п.п. и снижает стоимость на 38 % — накладные расходы адаптивной обвязки не окупаются на текущем пространстве задач, что согласуется с отрицательным ответом на RQ2. Замена правил на LLM-маршрутизатор экономит более половины стоимости при умеренной потере качества — сопутствующая находка, не воспроизводимая на семействе Qwen. Защитные правила переключения отдельным запуском не абляционировались; их вклад косвенно проявляется в метрике r_{guard} и в отсутствии осцилляционных режимов переключения во всех запусках серии.

4.9 Выводы по главе

Итоги серии: RQ1 подтвержден (класс задач определяет оптимальную статистику); RQ2 не подтвержден (адаптивная мета-топология не превосходит лучшую статистику ни по качеству, ни по стоимости), сопутствующая находка о двукратной экономии стоимости на исходной семье не воспроизводится

на Qwen; RQ3 подтвержден (HITL дает скромный прирост при неоднородности); RQ4 не подтвержден — направление эффекта совпадает на 4 из 4 корпусов на исходной семье, но при смене на Qwen знак меняется. Единственный устойчивый кросс-семейный вывод — зависимость оптимальной политики маршрутизации от семейства весов работника, разбираемая в главе 5.

5 Анализ результатов и обсуждение

Настоящая глава содержит обобщающий анализ экспериментальных результатов главы 4, последовательные ответы на четыре исследовательских вопроса, формулировку центрального вывода работы и обсуждение угроз валидности.

5.1 Ответы на исследовательские вопросы

RQ1. Зависимость оптимальной статической топологии от класса задач подтверждается — три разных победителя на четырех корпусах (Chain — HumanEval, Star — GSM8K, Debate — CommonGen и DABench). Лучшая глобальная статика (Chain, $q = 0,676$) уступает оракул-выбору под корпус ($q = 0,716$) на 4 п.п. — эта разность задает референс для E3.

RQ2. Гипотеза не подтверждается. Референс — оракул-выбор лучшей статистики для каждого корпуса ($q = 0,716$, $c = 0,014$); оракул адаптивной обвязки дает $q = 0,697$ (отставание 1,9 п.п.), правилowej и языковой — 5,3 и 6,6 п.п. ниже оракула обвязки. Самый дешевый режим (языковой) теряет 8,5 п.п. ради 30 % экономии стоимости. Причина — накладные расходы обвязки, не окупающиеся на корпусах с высоким качеством статистики. Положительный эффект адаптации концентрируется на CommonGen.

Сопутствующая находка — внутри адаптивного класса языковой маршрутизатор уступает правилowому 1,3 п.п. качества, будучи в 2,38 раза дешевле (интервал отношения стоимостей $[1,94, 2,94]$ исключает единицу); диапазон по корпусам 2,12–3,00 указывает на структурный характер. На Qwen соотношение коллапсирует до 1,11 с одновременным падением качества языкового режима на 30,6 п.п. — находка не воспроизводится.

RQ3. HITL дает направленный лифт +0,033 (+4,8 %) при неоднородности. Победители ролей по корпусам — Coordinator (HumanEval), Monitor (GSM8K), Peer (CommonGen, DABench); по топологиям — Star (Monitor/Reviewer, $q = 0,944$), Chain (Reviewer, $q = 0,677$), Hierarchical (Coordinator, $q = 0,842$), Debate (Peer, $q = 0,723$), Mesh (Peer, $q = 0,600$). Лифт выше 5 п.п. — на

Hierarchical×CommonGen, Chain×DABench, Mesh×DABench.

RQ4. Гипотеза не подтверждается. На исходной семье правилковой маршрутизатор роли дает +0,027, но CI $[-0,055, +0,110]$ покрывает нуль; на Qwen эффект становится $-0,081$. С идентичной DEFAULT_ROLE_TABLE правилковой маршрутизатор коллапсирует в разные роли на разных семьях — *координатор* в 98 % переходов на gpt-oss и *равноправный участник* в 94 % на Qwen — что указывает на зависимость «правилковой» политики от базовой модели через входы.

5.2 Главный нарратив исследования

Единственный эффект, прошедший бутстрап-валидацию на обеих семьях, — *зависимость оптимальной политики маршрутизации от семейства весов работника*. Он имеет три проявления.

Разворот Парето-победителя. На gpt-oss граница Парето — языковой маршрутизатор (двукратная экономия при паритете качества); на Qwen — правилковой (выше качество и дешевле на единицу качества), интервал разности качеств $[+0,233, +0,378]$ исключает нуль.

Схлопывание структуры стоимостей. На gpt-oss отношение стоимостей устойчиво 2,38; на Qwen коллапсирует до 1,11 с интервалом $[0,98, 1,25]$, покрывающим единицу — изменение порядка структурное, а не случайное.

Сдвиг распределения ролей. Та же таблица и тот же кодовый путь дают качественно разные роли в 94 % переходов между семьями — источник «разумности» адаптивных политик перекладывается с таблиц правил на поведение базовой модели.

Центральный вывод — адаптивные политики маршрутизации в MAS LLM, выглядящие независимыми от базовой модели через таблицы или промпты, фактически зависят от нее через свои входы. Заявления об универсальности без кросс-семейной валидации эпистемологически слабы.

5.3 Сопоставление с гипотезами

Сопоставление полученных результатов с гипотезами, сформулированными в разделе 2, приведено в таблице 5.1.

Таблица 5.1. Сопоставление гипотез работы и эмпирических результатов

Гипотеза	Эмпирический статус	Уточнение
H1: класс задач определяет лучшую статическую топологию	Подтверждена	Победители на 4 корпусах различаются.
H2: адаптивная мета-топология превосходит лучшую статическую конфигурацию под корпус	Не подтверждена	Сопутствующая находка о двукратной экономии стоимости внутри адаптивного класса не отвечает на H2 и не подтверждена кросс-семейной валидацией.
H3: подключение человека увеличивает среднее качество	Подтверждена	Лифт +0,033, неоднородно по ячейкам.
H4: совместная адаптация топологии и роли превосходит фиксированную роль	Не подтверждена	На исходной семье направление +0,027, доверительный интервал покрывает нуль; на семье Qwen знак меняется.

5.4 Угрозы валидности и ограничения

Работа имеет ряд ограничений.

Размер выборки — $n = 180$ запусков на ячейку E3/E4 недостаточен для устойчивой проверки эффектов меньше 5 п.п.; все направленные приросты укладываются в этот диапазон, и бутстрап-СІ покрывают нуль. Устойчивы только эффекты выше ± 25 п.п.

Кросс-семейная валидация — только две семьи (gpt-oss, Qwen); требуется минимум одна-две дополнительные (Claude, Gemini, LLaMA-4).

Смешение глубины рассуждений — интерфейс Qwen не принимает параметр `reasoning_effort`, что смешивает эффект семейства с эффектом глубины рассуждений; требуется серия с совпадающим параметром.

Симулятор человека — эмпирическая проверка соответствия симулятора и реального участника не выполнена (вынесена в E5); выводы об эффекте ролей относятся к симуляции.

Корпус задач — четыре открытых набора по одному на класс; обобщение на непокрытые классы (диалог, перевод, обобщение длинных документов) не проверено.

Судья — единственная модель из единственного семейства (gpt-4.1-mini); требуется параллельная оценка вторым судьей для исключения смещения.

Защитные правила — маршрутизатор роли коллапсирует в одну роль в 94–98 % переходов, фазовой адаптивности фактически нет; требуется ослабление правил в повторной серии.

5.5 Выводы по главе

RQ1 и RQ3 подтверждены, RQ2 и RQ4 — не подтверждены. Две сопутствующие находки (двукратная экономия стоимости языкового маршрутизатора и прирост качества от правилowej маршрутизации роли) не отвечают на исходные вопросы и при кросс-семейной валидации не воспроизводятся. Центральный вывод — зависимость оптимальной политики маршрутизации от семейства весов работника — формулируется как методологическое требование — любые претензии на универсальность адаптивных политик в MAS LLM требуют валидации не менее чем на двух семействах весов.

Авторский вклад

Авторский вклад настоящей работы сводится к четырем результатам.

Первый — программная инфраструктура исследования. В рамках работы с нуля разработан и опубликован под открытой лицензией MIT мульти-агентный фреймворк адаптивных топологий для мульти-агентных систем (от англ. Adaptive Topologies for Multi-agent systems, ATM), объединяющий пять статических коммуникационных топологий, адаптивную мета-топологию второго уровня, конечный автомат фаз, три режима маршрутизатора фаз и три режима маршрутизатора топологий, защитные правила переключения, шлюз взаимодействия с человеком с пятью ролями и маршрутизатором ролей, оракул-

маршрутизатор и подсистему наблюдаемости с двухкомпонентным хранилищем (реляционная база PostgreSQL и колоночный формат Apache Parquet). Кодовая база содержит 21 327 строк кода на языке Python, распределенных по 112 программным модулям внутри тринадцати функциональных слоев.

Второй — формальная модель и таксономия метрик. Введена формализация мульти-агентной системы как кортежа из шести компонентов с зависящей от фазы коммуникационной структурой (формулы 2.2 и 2.3). Введена таксономия из пяти адаптивно-специфичных метрик — число переключений топологии, доля заблокированных решений маршрутизатора, стоимостный вклад маршрутизатора, доля регрессий качества и разрыв до оракула (формула 2.4). Введена прокси-метрика когнитивной нагрузки для запусков с симулятором (формула 2.5). Совокупность метрик позволяет количественно характеризовать поведение динамически перестраиваемых мульти-агентных систем и сопоставлять их по единой шкале.

Третий — эмпирические результаты. Проведена воронка из четырех основных экспериментов и двух кросс-семейных подтверждающих запусков общим объемом 5175 запусков с нулевой долей отказов в финальных грид-сериях. Для каждого исследовательского вопроса получен явный количественный ответ — по RQ1 и RQ3 ответы положительные (универсального статического победителя нет; оптимальная роль человека зависит от корпуса и топологии), по RQ2 и RQ4 ответы отрицательные (ни адаптивная мета-топология, ни совместная адаптация топологии и роли не превосходят референс). Рядом с отрицательными ответами обнаружены две сопутствующие находки — двукратная экономия стоимости маршрутизатора на основе языковой модели внутри адаптивного класса и направленный прирост качества от правиловой маршрутизации роли в исходной семье моделей — которые не отвечают на исходно поставленные вопросы и при кросс-семейной валидации также не подтверждаются. Все головные эффекты сопровождаются непараметрическими бутстрап-доверительными интервалами.

Четвертый, ключевой — методологический вывод. Установлена зависимость оптимальной политики маршрутизации в адаптивных мульти-агентных

системах от семейства весов рабочего агента и продемонстрировано, что «правилковые» политики, выглядящие независимыми от модели через таблицы, фактически зависят от нее через свои входы (классификаторы фаз). На основании этого наблюдения сформулировано методологическое требование — любые претензии на универсальность адаптивных политик в мульти-агентных системах больших языковых моделей требуют валидации не менее чем на двух семействах весов рабочего агента. Это требование меняет статус кросс-семейной валидации с необязательного дополнения на обязательный шаг при исследовании адаптивных мульти-агентных систем.

Заключение

В работе проведена количественная оценка влияния выбора коммуникационной топологии, механизма ее адаптации и позиции человека на эффективность решения задач разных классов MAS LLM. Цель достигнута — получены количественные характеристики для пяти статических топологий, адаптивной мета-топологии в трех режимах маршрутизации, пяти ролей человека и их адаптивной комбинации; выполнено 5175 запусков на четырех корпусах задач с суммарной стоимостью \$87.

RQ1 (от англ. Research Question) подтвержден — универсальной статикодоминатора нет, победители на четырех корпусах различаются (Chain — HumanEval, Star — GSM8K, Debate — CommonGen и DABench). RQ3 подтвержден — HITL дает прирост +0,033 при неоднородности по ячейкам. RQ2 не подтвержден — адаптивная мета-топология не превосходит лучшую статику под корпус ни по качеству, ни по стоимости; сопутствующая находка о двукратной экономии стоимости маршрутизатора на основе языковой модели (CI [1,94, 2,94]) на исходной семье не воспроизводится на Qwen. RQ4 не подтвержден — прирост +0,027 имеет CI [−0,055, +0,110], на Qwen знак меняется на −0,081.

Ключевой результат — кросс-семейная неустойчивость адаптивных политик маршрутизации в MAS LLM. Таблицы правил и архитектуры промптов, выглядящие независимыми от базовой модели, фактически зависят от нее через входы — фазовые классификаторы получают разные сигналы от работников разных семейств и относят те же ситуации к разным фазам. Это формирует методологическое требование к кросс-семейной валидации как обязательному шагу исследования.

Ограничения — $n = 180$ запусков на ячейку недостаточно для эффектов меньше 5 п.п.; валидация на двух семействах; параметр глубины рассуждений смешан с эффектом семейства; человек моделируется языковой моделью; корпус не покрывает ряд классов (диалог, перевод, обобщение длинных текстов); судья — единственная модель.

Направления дальнейших исследований

Прямое продолжение работы образует шесть взаимодополняющих направлений. Первое — расширение кросс-семейной валидации на три и более семейства весов рабочего агента (Anthropic Claude, Google Gemini, Meta LLaMA-4) с приведением параметра глубины рассуждений к единой шкале, что устранил смещение эффектов семейства и режима работы агента, ставшее основным ограничением подтверждающих запусков настоящей работы. Это позволит проверить, является ли наблюдаемая кросс-семейная неустойчивость политик маршрутизации универсальным свойством адаптивных систем или специфичной для конкретной пары семейств. Второе — валидация симулятора человека на реальных участниках по схеме планируемого эксперимента E5 с использованием шкалы NASA-TLX (от англ. NASA Task Load Index — индекс когнитивной нагрузки NASA), латино-квадратного дизайна на восьми-двенадцати участниках и отобранного подмножества из двенадцати задач, что позволит численно оценить корреляцию между симулированной и фактической оценкой когнитивной нагрузки. Третье — разработка семейно-осознанного маршрутизатора, в котором политика выбора топологии и роли явно учитывает семейство весов рабочего агента, например, через предварительное дообучение маршрутизатора на трассах целевой модели или через переключение между семейно-специфичными политиками в зависимости от активного работника. Четвертое — расширение корпуса задач на непокрытые классы (диалог, перевод, обобщение длинных документов, мультимодальные задачи) и проверка обобщения результатов воронки на новые классы. Пятое — ослабление защитных правил переключения и проведение повторной серии E4 с целью выяснить, является ли наблюдаемое коллапсирование маршрутизатора роли в единственную роль артефактом текущих настроек или фундаментальным свойством адаптивной маршрутизации в исследуемом стеке. Шестое — параллельная оценка качества решений двумя судьями из разных семейств весов, что позволит численно отделить эффект адаптации от возможного смещения единственного судьи.

Библиографический список

- [1] A survey on large language model based autonomous agents / L. Wang, C. Ma, X. Feng [и др.] // *Frontiers of Computer Science*. – 2024. – Vol. 18, no. 6. – P. 186345.
- [2] AgentBench: Evaluating LLMs as agents [Электронный ресурс] / X. Liu, H. Yu, H. Zhang [и др.] // *arXiv preprint 2308.03688*. – 2023. – URL: <https://arxiv.org/abs/2308.03688> (дата обращения 17.05.2026).
- [3] AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors [Электронный ресурс] / W. Chen, Y. Su, J. Zuo [и др.] // *arXiv preprint 2308.10848*. – 2023. – URL: <https://arxiv.org/abs/2308.10848> (дата обращения: 17.05.2026).
- [4] AutoGen: Enabling next-gen LLM applications via multi-agent conversation [Электронный ресурс] / Q. Wu, G. Bansal, J. Zhang [и др.] // *arXiv preprint 2308.08155*. – 2023. – URL: <https://arxiv.org/abs/2308.08155> (дата обращения: 17.05.2026).
- [5] Benjamini Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing / Y. Benjamini, Y. Hochberg // *Journal of the Royal Statistical Society. Series B*. – 1995. – Vol. 57, no. 1. – P. 289–300.
- [6] CAMEL: Communicative agents for «mind» exploration of large language model society / G. Li, H. Hammoud, H. Itani, D. Khizbullin, B. Ghanem // *Advances in Neural Information Processing Systems*. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 51991–52008.
- [7] Chase H. LangChain: Building applications with LLMs through composability [Электронный ресурс] / H. Chase. – 2023. – URL: <https://github.com/langchain-ai/langchain> (дата обращения: 17.05.2026).
- [8] ChatDev: Communicative agents for software development / C. Qian, W. Liu, H. Liu [и др.] // *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. – Stroudsburg, PA: ACL, 2024. – P. 15174–15186.
- [9] Chen L. FrugalGPT: How to use large language models while reducing cost and improving performance [Электронный ресурс] / L. Chen, M. Zaharia, J. Zou // *Transactions on Machine Learning Research*. – 2024. – URL: <https://openreview.net/forum?id=cSimKw5p6R> (дата обращения: 17.05.2026).
- [10] Christiano P. F. Deep reinforcement learning from human preferences / P. F. Christiano, J. Leike, T. Brown [и др.] // *Advances in Neural Information Processing Systems*. – Red Hook, NY: Curran Associates, 2017. – Vol. 30. – P. 4299–4307.
- [11] CommonGen: A constrained text generation challenge for generative commonsense reasoning / B. Y. Lin, W. Zhou, M. Shen [и др.] // *Findings of the Association for Computational Linguistics (EMNLP 2020)*. – Stroudsburg, PA: ACL, 2020. – P. 1823–1840.
- [12] Cohen J. Statistical power analysis for the behavioral sciences / J. Cohen. – 2nd ed. – Hillsdale, NJ: Lawrence Erlbaum Associates, 1988. – 567 p.
- [13] Colvin S. Pydantic: Data validation using Python type hints [Электронный ресурс] / S. Colvin [и др.]. – 2024. – URL: <https://docs.pydantic.dev/> (дата обращения: 17.05.2026).
- [14] Efron B. Bootstrap methods: another look at the jackknife / B. Efron // *The Annals of Statistics*. – 1979. – Vol. 7, no. 1. – P. 1–26.
- [15] Evaluating large language models trained on code [Электронный ресурс] / M. Chen, J. Tworek, H. Jun [и др.] // *arXiv preprint 2107.03374*. – 2021. – URL: <https://arxiv.org/abs/2107.03374> (дата обращения: 17.05.2026).
- [16] G-Designer: Architecting multi-agent communication topologies via graph neural networks [Электронный ресурс] / Y. Zhang, K. Xu, Y. Li, Z. Liu, D. Shen //

- arXiv preprint 2410.11782. – 2024. – URL: <https://arxiv.org/abs/2410.11782> (дата обращения 17.05.2026).
- [17] GPTSwarm: Language agents as optimizable graphs / M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, J. Schmidhuber // Proceedings of the International Conference on Machine Learning (ICML 2024). – Cambridge, MA: PMLR, 2024. – P. 62556–62583.
- [18] Hart S. G. Development of NASA-TLX (Task Load Index): results of empirical and theoretical research / S. G. Hart, L. E. Staveland // Human Mental Workload / ed. by P. A. Hancock, N. Meshkati. – Amsterdam: North-Holland, 1988. – P. 139–183.
- [19] HellaSwag: Can a machine really finish your sentence? / R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, Y. Choi // Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019). – Stroudsburg, PA: ACL, 2019. – P. 4791–4800.
- [20] Horvitz E. Principles of Mixed-Initiative User Interfaces / E. Horvitz // Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99). – New York: ACM, 1999. – P. 159–166.
- [21] Improving factuality and reasoning in language models through multiagent debate [Электронный ресурс] / Y. Du, S. Li, A. Torralba, J. Tenenbaum, I. Mordatch // arXiv preprint 2305.14325. – 2023. – URL: <https://arxiv.org/abs/2305.14325> (дата обращения: 17.05.2026).
- [22] InfiAgent-DABench: Evaluating agents on data analysis tasks [Электронный ресурс] / X. Hu, Z. Zhao, S. Wei [и др.] // arXiv preprint 2401.05507. – 2024. – URL: <https://arxiv.org/abs/2401.05507> (дата обращения: 17.05.2026).
- [23] Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation / J. Liu, C. S. Xia, Y. Wang, L. Zhang // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 21558–21572.
- [24] LangGraph: A framework for building stateful, multi-actor applications with LLMs [Электронный ресурс] / LangChain AI. – Электрон. дан. – 2024. – URL: <https://langchain-ai.github.io/langgraph/> (дата обращения 17.05.2026).
- [25] LangSmith: Observability and evaluation for LLM applications [Электронный ресурс] / LangChain AI. – Электрон. дан. – 2024. – URL: <https://docs.smith.langchain.com/> (дата обращения: 17.05.2026).
- [26] Lin C.-Y. ROUGE: A package for automatic evaluation of summaries / C.-Y. Lin // Text Summarization Branches Out: Proceedings of the ACL-04 Workshop. – Stroudsburg, PA: ACL, 2004. – P. 74–81.
- [27] LiveCodeBench: Holistic and contamination-free evaluation of large language models for code [Электронный ресурс] / N. Jain, K. Han, A. Gu [и др.] // arXiv preprint 2403.07974. – 2024. – URL: <https://arxiv.org/abs/2403.07974> (дата обращения: 17.05.2026).
- [28] Liu Z. Dynamic LLM-agent network: an LLM-agent collaboration framework with agent team optimization [Электронный ресурс] / Z. Liu, Y. Zhang, P. Li, Y. Liu, D. Yang // arXiv preprint 2310.02170. – 2023. – URL: <https://arxiv.org/abs/2310.02170> (дата обращения: 17.05.2026).
- [29] Magentic-One: A generalist multi-agent system for solving complex tasks [Электронный ресурс] / A. Fourney, G. Bansal, H. Mozannar [и др.] // arXiv preprint 2411.04468. – 2024. – URL: <https://arxiv.org/abs/2411.04468> (дата обращения: 17.05.2026).
- [30] Measuring mathematical problem solving with the MATH dataset / D. Hendrycks, C. Burns, S. Kadavath [и др.] // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2021. – Vol. 34. – P. 2280–2292.

- [31] MetaGPT: Meta programming for a multi-agent collaborative framework / S. Hong, M. Zhuge, J. Chen [и др.] // Proceedings of the International Conference on Learning Representations (ICLR 2024). – 2024.
- [32] Mixture-of-Agents enhances large language model capabilities / J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, J. Zou // Proceedings of the Conference on Language Modeling (COLM 2024). – 2024.
- [33] MMLU-Pro: A more robust and challenging multi-task language understanding benchmark [Электронный ресурс] / Y. Wang, X. Ma, G. Zhang [и др.] // arXiv preprint 2406.01574. – 2024. – URL: <https://arxiv.org/abs/2406.01574> (дата обращения: 17.05.2026).
- [34] Mosqueira-Rey E. Human-in-the-loop machine learning: a state of the art / E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, Á. Fernández-Leal // Artificial Intelligence Review. – 2023. – Vol. 56, no. 4. – P. 3005–3054.
- [35] NetSafe: Exploring the topological safety of multi-agent networks against adversarial attacks [Электронный ресурс] / M. Yu, S. Wang, G. Zhang, H. Li // arXiv preprint 2410.15686. – 2024. – URL: <https://arxiv.org/abs/2410.15686> (дата обращения: 17.05.2026).
- [36] Ouyang L. Training language models to follow instructions with human feedback / L. Ouyang, J. Wu, X. Jiang [и др.] // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2022. – Vol. 35. – P. 27730–27744.
- [37] ReAct: Synergizing reasoning and acting in language models / S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, Y. Cao // Proceedings of the International Conference on Learning Representations (ICLR 2023). – 2023.
- [38] Reflexion: Language agents with verbal reinforcement learning / N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 8634–8652.
- [39] RouteLLM: Learning to route LLMs with preference data [Электронный ресурс] / I. Ong, A. Almahairi, V. Wu [и др.] // arXiv preprint 2406.18665. – 2024. – URL: <https://arxiv.org/abs/2406.18665> (дата обращения: 17.05.2026).
- [40] Scaling large-language-model-based multi-agent collaboration [Электронный ресурс] / C. Qian, Z. Ye, W. Liu [и др.] // arXiv preprint 2406.07155. – 2024. – URL: <https://arxiv.org/abs/2406.07155> (дата обращения: 17.05.2026).
- [41] Self-Refine: Iterative refinement with self-feedback / A. Madaan, N. Tandon, P. Gupta [и др.] // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 46534–46594.
- [42] Tipping the dominos: Topology-aware multi-hop attacks on LLM-based multi-agent systems [Электронный ресурс] / R. Liang, J. Ye, Z. Chen, Q. Zhu // arXiv preprint 2412.04129. – 2024. – URL: <https://arxiv.org/abs/2412.04129> (дата обращения: 17.05.2026).
- [43] Training verifiers to solve math word problems [Электронный ресурс] / K. Cobbe, V. Kosaraju, M. Bavarian [и др.] // arXiv preprint 2110.14168. – 2021. – URL: <https://arxiv.org/abs/2110.14168> (дата обращения: 17.05.2026).
- [44] Tree of Thoughts: Deliberate problem solving with large language models / S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, K. Narasimhan // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 11809–11822.

Приложение А Функциональная схема системы

В настоящем приложении приведена функциональная схема программной системы, описанной в главе 3. Схема изображает движение управления и данных от поступления входных данных эксперимента до получения результирующих метрик и журналов. На рисунке А.1 двойная красная рамка отмечает авторские компоненты — слой адаптации, оракул-маршрутизатор и шлюз взаимодействия с человеком (внутри блока «Внешние службы»). Штриховые красные стрелки — две обратные связи: оракульная таблица возвращается в маршрутизатор топологий, HITL-сигналы поступают от человека к слою адаптации.

Пояснение к схеме

Поток выполнения начинается с поступления конфигурации эксперимента в формате YAML и описания задачи $\mathcal{T}$ из выбранного корпуса. Оркестратор эксперимента (`atm.experiment`) инициализирует общее состояние графа и передает управление слою адаптации. Маршрутизатор фаз принимает решение о текущей фазе решения, передает результат маршрутизатору топологий, который выбирает активную коммуникационную топологию. В работе одновременно действует ровно одна из пяти базовых топологий, а адаптивный режим реализуется через последовательное переключение между ними по решению маршрутизатора.

Активная топология определяет порядок активации агентов и протокол обмена сообщениями между ними. Любая из шести ролей агентов (планировщик, исследователь, исполнитель, критик, дебатер, координатор) может быть подключена к любой топологии в соответствии с правилами раздела 2.2. Агенты обращаются за внешними службами — инструментами и Docker-песочницей для исполнения кода, оболочкой над языковыми моделями для генерации текста, шлюзом взаимодействия с человеком.

Все события системы (вызовы языковых моделей, выставление сигналов, переходы между фазами, переключения топологий, обращения к челове-

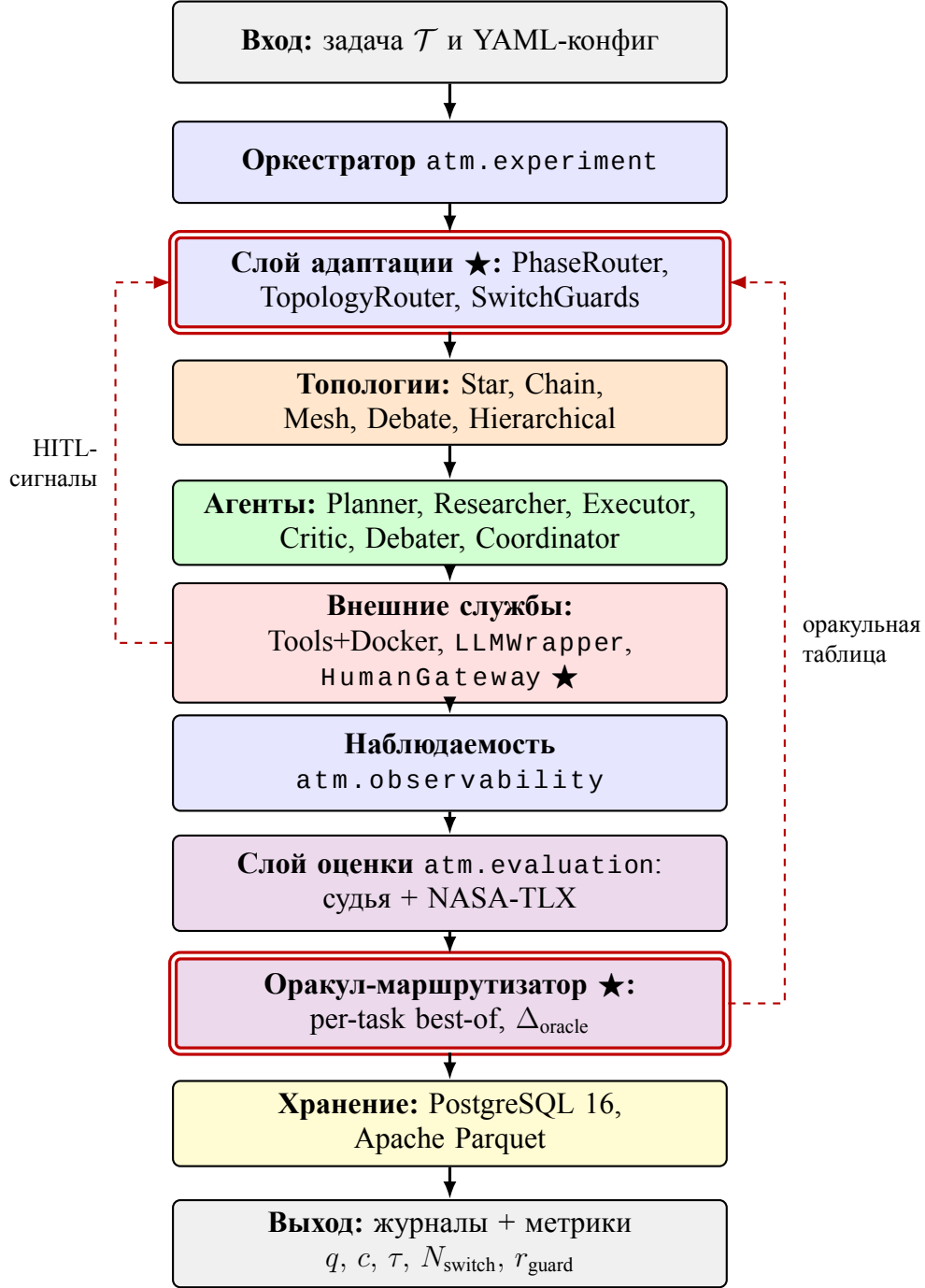


Рисунок А.1. Функциональная схема программной системы. Сплошные стрелки — прямой поток управления от входа к выходу. Штриховые красные стрелки — обратные связи: оракульная таблица возвращается в слой адаптации (маршрутизатор топологий), НITL-сигналы поступают от человека (внутри блока внешних служб) к слою адаптации. Двойная красная рамка обозначает авторские компоненты — слой адаптации, оракул-маршрутизатор и шлюз HumanGateway внутри блока внешних служб (отмечен ★). Полный перечень заимствованных и авторских компонентов с источниками приведен в таблице 3.2.

ку, срабатывания защитных правил маршрутизатора) перехватываются подсистемой наблюдаемости и направляются в два хранилища — реляционную базу для метаданных и колоночные файлы Apache Parquet для объемных журналов. На финальной стадии результаты запуска становятся доступными для постобработки слою анализа `atm.analysis`.

Обратный канал от шлюза человека к слою адаптации обозначает поступление управляющих сигналов от участника-человека. Через этот канал человек, действующий в роли координатора или наблюдателя, может форсировать переключение топологии или досрочно завершить фазу проверки. Штриховая связь от оракула к слою адаптации обеспечивает возврат оракульной таблицы (`per-task best-of`) для расчета Δ_{oracle} .

Приложение Б Глоссарий

В приложении приведены определения ключевых терминов работы, сгруппированные по тематическим блокам.

Мульти-агентные системы и языковые модели

LLM (Large Language Model) — глубокая нейронная сеть, обученная задаче языкового моделирования с дополнительной настройкой под инструкции. **MAS** (Multi-Agent System) — программная система из нескольких автономных агентов. **Агент** — программная сущность с фиксированной ролью, системной подсказкой, набором инструментов и собственным состоянием. **Скрэтчпад** — локальный контекст агента со скользящим окном истории и автоматическим сжатием при переполнении. **Судья** (LLM-as-judge) — внешняя языковая модель из другого семейства весов, оценивающая качество ответа с самосогласованностью. **Симулятор человека** — языковая модель в роли участника с подсказкой, зависящей от роли. **Бюджет вычислений** — трехуровневый ограничитель стоимости вызовов (на вызов / запуск / грид-эксперимент).

Топологии, фазы и маршрутизаторы

Коммуникационная топология — направленный граф допустимых маршрутов обмена сообщениями между агентами. **Star / Chain / Mesh / Debate / Hierarchical** — пять базовых топологий (центр-периферия, конвейер, полносвязная, два оппонента и судья, двухуровневая иерархия). **Адаптивная мета-топология** — конфигурация с выбором активной базовой топологии на каждой итерации в зависимости от фазы и сигналов; авторская разработка. **FSM** (Finite-State Machine) — конечный автомат фаз решения (планирование, исполнение, проверка, завершение). **PhaseRouter, TopologyRouter, RoleRouter** — маршрутизаторы фаз, топологий и ролей человека в трех режимах — правилый, на основе языковой модели и оракульный. **SwitchGuards** — защитные правила переключения против частого осцилляционного режима (min_dwll, cooldown, лимиты за фазу и запуск).

Человек в контуре управления

HITL (Human-in-the-Loop) — парадигма обращения автоматизированной системы к человеку за решением, оценкой или подтверждением. **HumanGateway** — протокол шлюза с идемпотентностью по паре идентификаторов запуска и запроса. **Пять ролей человека** — координатор (управляет планом), рецензент (проверяет результаты), судья (выносит вердикт), равноправный участник (вносит сообщения наравне с агентами), наблюдатель (вмешивается только в критических случаях). **NASA-TLX** — стандартная шкала субъективной оценки

когнитивной нагрузки по шести подшкалам.

Метрики

q — агрегированное качество в $[0, 1]$, формула зависит от класса задач. c — суммарная стоимость вызовов в долларах за запуск. τ — полное время запуска по настенным часам. N_{switch} — число фактических переключений активной топологии за запуск. r_{guard} — доля решений маршрутизатора, заблокированных защитными правилами. γ — доля бюджета на собственные вызовы маршрутизатора. r_{hurt} — доля задач с качеством хуже лучшей статической конфигурации. Δ_{oracle} — разрыв между оракул-выбором лучшей статической топологии для каждого корпуса (per-task best-of) и фактическим маршрутизатором. L_{cog} — прокси-метрика когнитивной нагрузки из числа обращений, объема контекста и задержки ответа симулятора.

Инфраструктура и эксперимент

LangGraph — библиотека Python для построения направленных графов состояния с чекпойнтером. **Apache Parquet** — колоночный формат хранения структурированных данных, используется для журналов взаимодействия. **Запуск (run)** — один цикл выполнения мульти-агентной системы; элементарная единица измерения. **Грид-эксперимент** — серия запусков по декартову произведению параметров. **Воронка экспериментов** — последовательность экспериментов с сужением пространства конфигураций по результатам предыдущих. **Подтверждающий запуск** — эксперимент на отличной языковой модели для проверки независимости выводов от семейства весов. **Бутстрап** — непараметрический метод оценивания распределения статистики через ресэмплирование; в работе применяется метод процентилей с десятью тысячами ресэмплов на уровне 95 %. **Поправка Бенджамини–Хохберга** — процедура контроля доли ложных открытий при множественном сравнении. **Коэффициент Коэна (d)** — стандартизированная мера величины различия двух средних.

Приложение В Состав программного репозитория

Состав репозитория фиксирован по идентификатору фиксации приложения Г.

Корневая структура

src/atm/ — основной пакет (13 слоев). **conf/** — YAML-конфигурации экспериментов и компонентов. **scripts/** — скрипты обслуживания. **notebooks/** — шаблон Jupyter-ноутбука для постобработки. **pyproject.toml** — декларация пакета и точки входа **atm**. **README.md** — инструкция по сборке.

Слой пакета **src/atm/**

atm.core — состояние и базовые типы. **atm.llm** — оболочка провайдеров, ценообразование, бюджеты, FakeLLM. **atm.tools** — реестр инструментов и Docker-песочница. **atm.agents** — класс Agent, пять ролей и координатор. **atm.topology** — шесть топологий поверх протокола Topology. **atm.phases** — FSM фаз, маршрутизатор топологий, защитные правила, ворота передачи состояния. **atm.human** — HITL: HumanGateway, шлюзы, маршрутизатор ролей, политика тайм-аутов. **atm.storage** — ORM, асинхронные сессии PostgreSQL, писатель Parquet, чекпойнтер, миграции alembic/. **atm.observability** — перехватчик событий LangGraph. **atm.tasks** — загрузчики и оценщики четырех корпусов. **atm.evaluation** — судья качества, оракулы, агрегатор NASA-TLX. **atm.experiment** — схемы Pydantic, OmegaConf, CLI на Typer. **atm.analysis** — агрегаторы метрик, графики, построитель оракула.

Конфигурации **conf/**

experiments/ — конфиги E1–E4 и подтверждающих запусков. **agents/**, **topology/**, **human/**, **model/**, **tasks/**, **oracle/**, **evaluation/**, **sandbox/** — параметры по соответствующим компонентам.

Приложение Г Открытый исходный код

В соответствии с пунктом 5.5 Правил подготовки и защиты ВКР полная программная реализация настоящего исследования размещена в открытом доступе на платформе GitHub.

Адрес репозитория

<https://github.com/cactus-tim/adaptive-topologies-mas>

Идентификатор фиксации

ветка `main`, короткий идентификатор коммита `058271a39a0e`, полный хеш:

`058271a39a0e4ecc0e3b47bd1d826c03fc33872b`.

Лицензия

MIT (файл `LICENSE`).

Сборка и воспроизведение

инструкция в `README.md`; конфигурации экспериментов главы 4 — в директории `conf/experiments/`; запуск одиночного запуска — `atm run --config <file>`, грид-эксперимента — `atm grid --config <file>`, где `<file>` — путь к YAML-конфигу.

Согласие автора на публикацию ВКР на портале НИУ ВШЭ (п. 5.6.3 Правил) оформлено QR-кодом системы «LMS-Антиплагиат» в составе пакета документов в ГЭК.